



Ovo je prikaz, mog projekta, kako da ga pokrenem, da vidim sta se desi

Tvoj projekt je standardna ASP.NET Core MVC aplikacija. Evo kako je pokrenuti:

Način 1 — Visual Studio (najlakše)

Budući da vidiš `.slnx` file u korijenu, otvori ga dvoklikom:^[1]

1. Otvori `PostMatchSummary.slnx` u Visual Studiju
2. Pritisni `F5` (s debuggerom) ili `Ctrl+F5` (bez debuggera)
3. Automatski se otvara browser na `https://localhost:PORT`

Način 2 — Terminal (dotnet CLI)

Otvori terminal i navigiraj do foldera gdje se nalazi `.csproj` file:^[2]

```
cd putanja\do\ASP.NET\PostMatchSummary
dotnet run
```

Nakon toga terminal ispiše nešto poput:

```
Now listening on: https://localhost:5001
```

Onda otvori taj URL u browseru.

Što očekivati

Iz strukture projekta vidiš `PostMatchController.cs` i `RiotServices.cs` (zadnji edit 14:52), što znači aplikacija vjerojatno poziva **Riot API** za dohvaćanje podataka o matchevima. Kad pokreneš, prikazat će se MVC stranice — `Home` i `PostMatch view`.^[3] ^[4]

Napomena: Ako `RiotServices.cs` zahtijeva API key, provjeri je li postavljen u `appsettings.json` ili kao environment varijabla — inače ćeš dobiti grešku pri dohvaćanju podataka.

Imaš li već API key od Riota postavljen u projektu, ili trebaš pomoć s tim?

✱

Microsoft Windows [Version 10.0.19044.6809]

(c) Microsoft Corporation. All rights reserved.

```
C:\Users\Matej\Desktop\ASP.NET\PostMatchSummary>dotnet run
Using launch settings from C:\Users\Matej\Desktop\ASP.NET
```

\PostMatchSummary\Properties\launchSettings.json...

Building...

C:\Users\Matej\Desktop\ASP.NET\PostMatchSummary\Models\Program.cs(1,1): error CS8802: Only one compilation unit can have top-level statements.

C:\Users\Matej\Desktop\ASP.NET\PostMatchSummary\Program.cs(1,1): error CS8802: Only one compilation unit can have top-level statements.

The build failed. Fix the build errors and run again.

C:\Users\Matej\Desktop\ASP.NET\PostMatchSummary>

ovo kad odem dotnet run u postmatchsummary folderu u asp.net projektu

Greška je jasna — imaš dva filea s top-level statements, a C# dozvoljava samo jedan u cijelom projektu: ^[26]

- PostMatchSummary/Program.cs ← pravi entry point aplikacije ✓
- PostMatchSummary/Models/Program.cs ← ovo uzrokuje konflikt ✗

Kako popraviti

Models/Program.cs vjerojatno treba biti klasa, ne standalone top-level kod. Otvori taj file i umotaj sadržaj u namespace i class:

Prije (što vjerojatno piše):

```
// neki kod direktno ovdje
```

Nakon (ispravno):

```
namespace PostMatchSummary.Models
{
    public class Program
    {
        // sadržaj filea ovdje
    }
}
```

Brzi koraci

1. Otvori Models/Program.cs u editoru
2. Provjeri što piše unutra — ako je to model (npr. League champion "Program" skin line), samo ga umotaj u klasu kao gore
3. Spremi i opet pokreni dotnet run

Savjet: Ime Program.cs u Models folderu je zbunjujuće — razmisli o preimenovanju u npr. ProgramChampion.cs ili sl. da izbjegneš ovakve konflikte u budućnosti.

Možeš paste-ati sadržaj tog Models/Program.cs filea da ti kažem točno što treba promijeniti?

✱

```
var matches = new List<Match>();
```

```
var match1 = new Match
{
    MatchId = "1",
    GameCreation = DateTime.Now,
    Duration = 1800,
    Players = new List<Player>()
};
```

```
var match2 = new Match
{
    MatchId = "2",
    GameCreation = DateTime.Now,
    Duration = 2000,
    Players = new List<Player>()
};
```

```
matches.Add(match1);
matches.Add(match2);
```

```
// svi match-evi duži od 1900 sekundi
var longMatches = matches
    .Where(m => m.Duration > 1900)
    .ToList();
```

```
// prvi match
var firstMatch = matches.FirstOrDefault();
```

```
// broj match-eva
var count = matches.Count();
```

ovo je u models

a ovo ispod u foleru postmatchsummary

```
var builder = WebApplication.CreateBuilder(args);
```

```
// Add services to the container.
builder.Services.AddControllersWithViews();
```

```
var app = builder.Build();
```

```
// Configure the HTTP request pipeline.
if (!app.Environment.IsDevelopment())
{
    app.UseExceptionHandler("/Home/Error");
    // The default HSTS value is 30 days. You may want to change this for production scenarios, see
https://aka.ms/aspnetcore-hsts.
    app.UseHsts();
}
```

```
app.UseHttpsRedirection();
app.UseRouting();
```

```
app.UseAuthorization();

app.MapStaticAssets();

app.MapControllerRoute(
    name: "default",
    pattern: "{controller=Home}/{action=Index}/{id?}")
    .WithStaticAssets();

app.Run();
```

Problem je što `Models/Program.cs` sadrži test/scratch kod koji nisi trebao commitati tamo — to su samo privremeni testovi LINQ querija, nije pravi model.

Rješenje — izbriši taj file

`Models/Program.cs` nema nikakve vrijednosti u projektu, to je bio samo test kod. Jednostavno ga obriši:

```
del "C:\Users\Matej\Desktop\ASP.NET\PostMatchSummary\Models\Program.cs"
```

Ili ga ručno obriši u File Exploreru / Visual Studiju.

Zašto ne treba wrap u klasu?

Taj kod nije model — nema smisla stavljati ga u klasu jer samo kreira `Match` objekte i radi LINQ filter. To je bio scratch/debug kod koji je slučajno završio u pogrešnom folderu.

Nakon brisanja

```
dotnet run
```

Trebalo bi raditi bez grešaka. Prava `Program.cs` u korijenu projekta (ona s `WebApplication.CreateBuilder`) ostaje i služi kao entry point aplikacije. ^[27]

✱

Microsoft Windows [Version 10.0.19044.6809]

(c) Microsoft Corporation. All rights reserved.

```
C:\Users\Matej\Desktop\ASP.NET\PostMatchSummary>dotnet run
Using launch settings from C:\Users\Matej\Desktop\ASP.NET
\PostMatchSummary\Properties\launchSettings.json...
Building...
```

```
C:\Users\Matej\Desktop\ASP.NET\PostMatchSummary\Program.cs(1,1): error CS8802: Only one
compilation unit can have top-level statements.
```

The build failed. Fix the build errors and run again.

```
C:\Users\Matej\Desktop\ASP.NET\PostMatchSummary>
```

nakon brisanja `program.cs` iz `models`a

Još uvijek postoji drugi file s top-level statementima. Primjeti da si imao `Main.cs` u `Models` folderu — to je vjerojatno novi krivac!

Provjeri Models/Main.cs

Otvori `Models/Main.cs` i paste-aj sadržaj ovdje. Gotovo sigurno ima sličan scratch kod kao i stari `Program.cs`.

Ako da, obriši i njega:

```
del "C:\Users\Matej\Desktop\ASP.NET\PostMatchSummary\Models\Main.cs"
```

Kako pronaći sve problematične fileove odjednom

Pokreni ovo u terminalu da vidiš sve `.cs` fileove koji imaju top-level statements (kod izvan klase/namespacea):

```
findstr /r /l /s "^var \|^using \|^new " "*.cs"
```

Generalno, svaki `.cs` file u `Models` folderu treba imati strukturu ovog oblika — ako file počinje direktno s `var`, `new`, `LINQ` itd. bez namespace/class — to je problem.

```
namespace PostMatchSummary.Models
{
    public class NeštoModel
    {
        // properties ovdje
    }
}
```

Što piše u `Main.cs`?

static void Main(string[] args)

```
{
    var service = new RiotService();

    var task = service.GetMatchAsync("EUW1_7796385639");

    task.Wait();

    var json = task.Result;

    Console.WriteLine(json);
}
```

To je bio test file za ručno testiranje `RiotService` — više ti ne treba u [ASP.NET](#) projektu jer se servis poziva kroz controller, ne kroz `Main`.

Rješenje — obriši Models/Main.cs

```
del "C:\Users\Matej\Desktop\ASP.NET\PostMatchSummary\Models\Main.cs"
```

Zašto je to bio problem?

`static void Main(string[] args)` je entry point konzolne aplikacije. C# kompajler ne zna koji entry point koristiti kada vidi i taj `Main` i top-level statements u `Program.cs` — pa baca CS8802 grešku.

ASP.NET Core projekt već ima svoj entry point u `Program.cs` (onaj `sWebApplication.CreateBuilder`), pa `Main.cs` nije potreban.

Nakon brisanja

```
dotnet run
```

Sada bi trebalo buildati i pokrenuti bez grešaka. ☑

Microsoft Windows [Version 10.0.19044.6809]

(c) Microsoft Corporation. All rights reserved.

```
C:\Users\Matej\Desktop\ASP.NET\PostMatchSummary>dotnet run
```

```
Using launch settings from C:\Users\Matej\Desktop\ASP.NET
```

```
\PostMatchSummary\Properties\launchSettings.json...
```

```
Building...
```

```
C:\Users\Matej\Desktop\ASP.NET\PostMatchSummary\Models\MatchSummary.cs(3,19): warning CS8618: Non-nullable property 'MatchId' must contain a non-null value when exiting constructor. Consider adding the 'required' modifier or declaring the property as nullable.
```

```
C:\Users\Matej\Desktop\ASP.NET\PostMatchSummary\Models\Player.cs(3,19): warning CS8618: Non-nullable property 'SummonerName' must contain a non-null value when exiting constructor. Consider adding the 'required' modifier or declaring the property as nullable.
```

```
C:\Users\Matej\Desktop\ASP.NET\PostMatchSummary\Models\Team.cs(6,25): warning CS8618: Non-nullable property 'Players' must contain a non-null value when exiting constructor. Consider adding the 'required' modifier or declaring the property as nullable.
```

```
C:\Users\Matej\Desktop\ASP.NET\PostMatchSummary\Models\Player.cs(4,19): warning CS8618: Non-nullable property 'Champion' must contain a non-null value when exiting constructor. Consider adding the 'required' modifier or declaring the property as nullable.
```

```
C:\Users\Matej\Desktop\ASP.NET\PostMatchSummary\Models\MatchSummary.cs(4,19): warning CS8618: Non-nullable property 'WinnerTeam' must contain a non-null value when exiting constructor. Consider adding the 'required' modifier or declaring the property as nullable.
```

```
C:\Users\Matej\Desktop\ASP.NET\PostMatchSummary\Models\Player.cs(9,17): warning CS8618: Non-nullable property 'Team' must contain a non-null value when exiting constructor. Consider adding the 'required' modifier or declaring the property as nullable.
```

```
C:\Users\Matej\Desktop\ASP.NET\PostMatchSummary\Models\Champion.cs(3,19): warning CS8618: Non-nullable property 'Name' must contain a non-null value when exiting constructor. Consider adding the 'required' modifier or declaring the property as nullable.
```

```
C:\Users\Matej\Desktop\ASP.NET\PostMatchSummary\Models\RiotResponse.cs(3,18): warning CS8618: Non-nullable property 'Info' must contain a non-null value when exiting constructor. Consider adding
```

the 'required' modifier or declaring the property as nullable.

C:\Users\Matej\Desktop\ASP.NET\PostMatchSummary\Models\Match.cs(3,19): warning CS8618: Non-nullable property 'MatchId' must contain a non-null value when exiting constructor. Consider adding the 'required' modifier or declaring the property as nullable.

C:\Users\Matej\Desktop\ASP.NET\PostMatchSummary\Models\Match.cs(7,25): warning CS8618: Non-nullable property 'Players' must contain a non-null value when exiting constructor. Consider adding the 'required' modifier or declaring the property as nullable.

C:\Users\Matej\Desktop\ASP.NET\PostMatchSummary\Models\Match.cs(8,23): warning CS8618: Non-nullable property 'Teams' must contain a non-null value when exiting constructor. Consider adding the 'required' modifier or declaring the property as nullable.

info: Microsoft.Hosting.Lifetime[14]

Now listening on: <http://localhost:5044>

info: Microsoft.Hosting.Lifetime[0]

Application started. Press Ctrl+C to shut down.

info: Microsoft.Hosting.Lifetime[0]

Hosting environment: Development

info: Microsoft.Hosting.Lifetime[0]

Content root path: C:\Users\Matej\Desktop\ASP.NET\PostMatchSummary

warn: Microsoft.AspNetCore.HttpsPolicy.HttpsRedirectionMiddleware[3]

Failed to determine the https port for redirect.

Projekt radi! ☑ Vidiš default ASP.NET stranicu na <http://localhost:5044> — to je uspjeh. ^[28]

Što su te poruke?

Warnings (upozorenja) — nisu greške, build je prošao i aplikacija se pokrenula. Možeš ih ignorirati za sada. Znače da tvoji modeli imaju `string` i `List` propertyje koji nisu inicijalizirani na `null-safe` način, npr:

```
// Trenutno (warning)
public string MatchId { get; set; }

// Opcija 1 - nullable (dodaj ?)
public string? MatchId { get; set; }

// Opcija 2 - required modifier
public required string MatchId { get; set; }
```

Failed to determine the https port — samo znači da HTTPS redirect ne radi jer nemaš SSL cert konfiguriran, ali HTTP na portu 5044 radi normalno.

Sljedeći korak

Tvoj projekt ima `PostMatchController` — idi na:

```
http://localhost:5044/PostMatch
```

To bi trebao biti tvoj glavni view s match summary podacima. Ako baca grešku, paste-aj je ovdje pa gledamo zajedno.



An unhandled exception occurred while processing the request.

InvalidOperationException: Unable to resolve service for type 'RiotService' while attempting to activate 'PostMatchController'.

Microsoft.Extensions.DependencyInjection.ActivatorUtilities.ThrowHelperUnableToResolveService(Type type, Type requiredBy)

Stack

Query

Cookies

Headers

Routing

InvalidOperationException: Unable to resolve service for type 'RiotService' while attempting to activate 'PostMatchController'.

Microsoft.Extensions.DependencyInjection.ActivatorUtilities.ThrowHelperUnableToResolveService(Type type, Type requiredBy)

lambda_method31(Closure, IServiceProvider, object[])

Microsoft.AspNetCore.Mvc.Controllers.ControllerFactoryProvider+<>c__DisplayClass6_0.

<CreateControllerFactory>g__CreateController|0(ControllerContext controllerContext)

Microsoft.AspNetCore.Mvc.Infrastructure.ControllerActionInvoker.Next(ref State next, ref Scope scope, ref object state, ref bool isCompleted)

Microsoft.AspNetCore.Mvc.Infrastructure.ControllerActionInvoker.InvokeInnerFilterAsync()

Microsoft.AspNetCore.Mvc.Infrastructure.ResourceInvoker.

<InvokeNextResourceFilter>g__Awaited|25_0(ResourceInvoker invoker, Task lastTask, State next, Scope scope, object state, bool isCompleted)

Microsoft.AspNetCore.Mvc.Infrastructure.ResourceInvoker.Rethrow(ResourceExecutedContextSealed context)

Microsoft.AspNetCore.Mvc.Infrastructure.ResourceInvoker.Next(ref State next, ref Scope scope, ref object state, ref bool isCompleted)

Microsoft.AspNetCore.Mvc.Infrastructure.ResourceInvoker.InvokeFilterPipelineAsync()

Microsoft.AspNetCore.Mvc.Infrastructure.ResourceInvoker.

<InvokeAsync>g__Awaited|17_0(ResourceInvoker invoker, Task task, IDisposable scope)

Microsoft.AspNetCore.Mvc.Infrastructure.ResourceInvoker.

<InvokeAsync>g__Awaited|17_0(ResourceInvoker invoker, Task task, IDisposable scope)

Microsoft.AspNetCore.Authorization.AuthorizationMiddleware.Invoke(HttpContext context)

Microsoft.AspNetCore.Authentication.AuthenticationMiddleware.Invoke(HttpContext context)

Microsoft.AspNetCore.Diagnostics.DeveloperExceptionPageMiddlewareImpl.Invoke(HttpContext context)

Show raw exception details

Greška je jasna — `RiotService` nije registriran u `Dependency Injection containeru`. `ASP.NET Core` ne zna kako ga kreirati kada ga `PostMatchController` traži.

Rješenje — dodaj u `Program.cs`

Otvori `Program.cs` (u korijenu projekta) i dodaj jednu liniju prije `builder.Build()`:

```
var builder = WebApplication.CreateBuilder(args);

// Add services to the container.
```

```
builder.Services.AddControllersWithViews();
builder.Services.AddHttpClient<RiotService>(); // ← DODAJ OVO

var app = builder.Build();
```

Koristi `AddHttpClient<RiotService>()` jer tvoj `RiotService` vjerojatno koristi `HttpClient` za pozivanje Riot API-ja.

Ako `RiotService` ne koristi `HttpClient`

Koristi jednostavniji oblik:

```
builder.Services.AddScoped<RiotService>();
```

Nakon izmjene

Spremi, zaustavi server (`Ctrl+C`), pa opet:

```
dotnet run
```

Iodi na `http://localhost:5044/PostMatch` — sada bi trebao raditi bez greške. Paste-aj sadržaj `RiotServices.cs` ako nisi siguran koja varijanta ti treba.

using LeagueOfLegendsApp.Services;

```
var builder = WebApplication.CreateBuilder(args);

// Add services to the container.
builder.Services.AddControllersWithViews();
builder.Services.AddHttpClient<RiotService>(); // ← DODAJ OVO

var app = builder.Build();

// Configure the HTTP request pipeline.
if (!app.Environment.IsDevelopment())
{
    app.UseExceptionHandler("/Home/Error");
    // The default HSTS value is 30 days. You may want to change this for production scenarios, see
    https://aka.ms/aspnetcore-hsts.
    app.UseHsts();
}

app.UseHttpsRedirection();
app.UseRouting();

app.UseAuthorization();

app.MapStaticAssets();

app.MapControllerRoute(
    name: "default",
```

```
pattern: "{controller=Home}/{action=Index}/{id?}")
.WithStaticAssets();
```

```
app.Run();
```

Problem je **pogrešan namespace** u using liniji! Projekt se zove `PostMatchSummary`, ali pišeš `LeagueOfLegendsApp.Services` — taj namespace ne postoji.

Ispravi using liniju

```
// Pogrešno:
using LeagueOfLegendsApp.Services;

// Ispravno:
using PostMatchSummary.Services;
```

Dakle cijeli `Program.cs` treba izgledati ovako:

```
using PostMatchSummary.Services;

var builder = WebApplication.CreateBuilder(args);

builder.Services.AddControllersWithViews();
builder.Services.AddHttpClient<RiotService>();

var app = builder.Build();

if (!app.Environment.IsDevelopment())
{
    app.UseExceptionHandler("/Home/Error");
    app.UseHsts();
}

app.UseHttpsRedirection();
app.UseRouting();
app.UseAuthorization();
app.MapStaticAssets();

app.MapControllerRoute(
    name: "default",
    pattern: "{controller=Home}/{action=Index}/{id?}")
    .WithStaticAssets();

app.Run();
```

Spremi, `Ctrl+C`, `dotnet run` i probaj opet `/PostMatch`. ☒

(c) Microsoft Corporation. All rights reserved.

```
C:\Users\Matej\Desktop\ASP.NET\PostMatchSummary>dotnet run
```

```
Using launch settings from C:\Users\Matej\Desktop\ASP.NET
```

```
\PostMatchSummary\Properties\launchSettings.json...
```

```
Building...
```

```
C:\Users\Matej\Desktop\ASP.NET\PostMatchSummary\Program.cs(1,24): error CS0234: The type or
```

namespace name 'Services' does not exist in the namespace 'PostMatchSummary' (are you missing an assembly reference?)

The build failed. Fix the build errors and run again.

C:\Users\Matej\Desktop\ASP.NET\PostMatchSummary>

Greška znači da `RiotServices.cs` nema dobar namespace deklariran unutar filea. Paste-aj sadržaj `Services/RiotServices.cs` da vidim što piše, ali najvjerojatnije nedostaje ili je pogrešan namespace.

Brzi fix — provjeri `RiotServices.cs`

Otvori `Services/RiotServices.cs` i provjeri je li na vrhu filea:

```
namespace PostMatchSummary.Services // ← mora točno ovako pisati
{
    public class RiotService
    {
        // ...
    }
}
```

Ako piše nešto drugo (npr. `LeagueOfLegendsApp.Services`, ili uopće nema namespace), ispravi na `PostMatchSummary.Services`.

Alternativno rješenje — bez using

Ako ne želiš dirati `RiotServices.cs`, možeš ukloniti `using` liniju iz `Program.cs` i koristiti puni naziv:

```
// Makni: using PostMatchSummary.Services;

builder.Services.AddHttpClient<PostMatchSummary.Services.RiotService>();
```

Paste-aj sadržaj `RiotServices.cs` pa ćemo točno vidjeti što treba promijeniti.

evo ti sve pa mi reci

Vidim problem! `RiotServices.cs` nema **nikakav namespace** — klasa je u globalnom namespace-u, a ti pokušavaš koristiti `PostMatchSummary.Services` koji ne postoji.^[29]

Imaš dvije opcije:

Opcija 1 — Dodaj namespace u `RiotServices.cs` (preporučeno)

Otvori `Services/RiotServices.cs` i dodaj namespace na vrh:

```
using System.Net.Http;
using System.Threading.Tasks;

namespace PostMatchSummary.Services // ← DODAJ OVO
{
    public class RiotService
    {
        private string apiKey = "RGAPI-...";
```

```

        public async Task<string> GetMatchAsync(string matchId)
        {
            using var client = new HttpClient();
            client.DefaultRequestHeaders.Add("X-Riot-Token", apiKey);
            string url = $"https://europe.api.riotgames.com/lol/match/v5/matches/{matchId}";
            var response = await client.GetAsync(url);
            return await response.Content.ReadAsStringAsync();
        }
    }
}

```

Opcija 2 — Makni using iz Program.cs

Ako ne želiš dirati `RiotServices.cs`, u `Program.cs` makni `using` i koristi `AddScoped`:

```

// Makni: using PostMatchSummary.Services;

builder.Services.AddScoped<RiotService>(); // radi jer je u global namespace-u

```

⚠ **Važno:** Tvoj API key je vidljiv u kodu (RGAPI-603eb17f-...) — taj key je vjerojatno već expiran jer Riot Development keyevi traju 24h, ali svakako ga makni iz koda i stavi u `appsettings.json`.^[29]

Preporučam Opciju 1 — dodaj namespace, to je čistije. Nakon toga `dotnet run` bi trebao proći. ☒

✱

ajde mi samo daj full klase napisane

Evo svih fileova ispunjenih s pravilnim namespaceima:

Program.cs

```

using PostMatchSummary.Services;

var builder = WebApplication.CreateBuilder(args);

builder.Services.AddControllersWithViews();
builder.Services.AddHttpClient<RiotService>();

var app = builder.Build();

if (!app.Environment.IsDevelopment())
{
    app.UseExceptionHandler("/Home/Error");
    app.UseHsts();
}

app.UseHttpsRedirection();
app.UseRouting();
app.UseAuthorization();
app.MapStaticAssets();

app.MapControllerRoute(
    name: "default",
    pattern: "{controller=Home}/{action=Index}/{id?}")

```

```
.WithStaticAssets();

app.Run();
```

Services/RiotServices.cs

```
using System.Net.Http;
using System.Threading.Tasks;

namespace PostMatchSummary.Services
{
    public class RiotService
    {
        private readonly HttpClient _httpClient;
        private string apiKey = "TVOJ-API-KEY-OVDJE";

        public RiotService(HttpClient httpClient)
        {
            _httpClient = httpClient;
        }

        public async Task<string> GetMatchAsync(string matchId)
        {
            _httpClient.DefaultRequestHeaders.Remove("X-Riot-Token");
            _httpClient.DefaultRequestHeaders.Add("X-Riot-Token", apiKey);
            string url = $"https://europe.api.riotgames.com/lol/match/v5/matches/{matchId}";
            var response = await _httpClient.GetAsync(url);
            return await response.Content.ReadAsStringAsync();
        }
    }
}
```

Controllers/PostMatchController.cs

```
using Microsoft.AspNetCore.Mvc;
using PostMatchSummary.Services;
using System.Threading.Tasks;

namespace PostMatchSummary.Controllers
{
    public class PostMatchController : Controller
    {
        private readonly RiotService _riotService;

        public PostMatchController(RiotService riotService)
        {
            _riotService = riotService;
        }

        public async Task<IActionResult> Index(string matchId)
        {
            var data = await _riotService.GetMatchAsync(matchId);
            return View("Index", data);
        }
    }
}
```

```
    }  
}
```

Models/Match.cs

```
namespace PostMatchSummary.Models  
{  
    public class Match  
    {  
        public string MatchId { get; set; } = string.Empty;  
        public DateTime GameCreation { get; set; }  
        public int Duration { get; set; }  
        public List<Player> Players { get; set; } = new();  
        public List<Team> Teams { get; set; } = new();  
    }  
}
```

Models/Player.cs

```
namespace PostMatchSummary.Models  
{  
    public class Player  
    {  
        public string SummonerName { get; set; } = string.Empty;  
        public string Champion { get; set; } = string.Empty;  
        public int Kills { get; set; }  
        public int Deaths { get; set; }  
        public int Assists { get; set; }  
        public Team Team { get; set; } = new();  
    }  
}
```

Models/Team.cs

```
namespace PostMatchSummary.Models  
{  
    public class Team  
    {  
        public int TeamId { get; set; }  
        public bool Win { get; set; }  
        public List<Player> Players { get; set; } = new();  
    }  
}
```

Models/MatchSummary.cs

```
namespace PostMatchSummary.Models  
{  
    public class MatchSummary  
    {  
        public string MatchId { get; set; } = string.Empty;
```

```

        public string WinnerTeam { get; set; } = string.Empty;
        public int TotalKills { get; set; }
    }
}

```

Models/RiotResponse.cs

```

namespace PostMatchSummary.Models
{
    public class RiotResponse
    {
        public object? Info { get; set; }
    }
}

```

Zamijeni sve fileove ovim sadržajem, pa `dotnet run`. Ključne promjene su: svi modeli dobili namespace `PostMatchSummary.Models`, `RiotService` dobio namespace `PostMatchSummary.Services` i prima `HttpClient` kroz DI umjesto da ga kreira sam.^[47]

✱

I dobro i gdje dobijem ispis traženih podataka

Moraš otići na URL s `matchId` parametrom u query stringu:^[56]

```
http://localhost:5044/PostMatch?matchId=EUW1_7796385639
```

`PostMatchController.Index` prima `matchId` kao parametar iz URL-a, šalje request na Riot API i vraća raw JSON koji se prikazuje u `<pre>` tagu na stranici.^[57]

Kako dobiti validan matchId?

Riot API match ID-evi izgledaju ovako: `EUW1_` + broj. Možeš ih dobiti:

1. **Riot Developer Portal** — <https://developer.riotgames.com> → API Explorer
2. **Direktno API pozivom** za nekog igrača:

```
https://europe.api.riotgames.com/lol/match/v5/matches/by-puuid/{PUUID}/ids
```

3. **OP.GG** ili **League of Graphs** — otvori neki match i iz URL-a izvuci ID

Što ćeš vidjeti

Na stranici će se ispisati sirovi JSON od Riot API-ja — sve informacije o matchu (igrači, championsi, KDA, trajanje...). Ako dobiješ `403 Forbidden`, znači da ti je **API key** expirao — Riot development keyevi traju samo 24h pa trebaš generirati novi na developer.riotgames.com.

✱

Match Data

```
{"status":{"message":"Forbidden","status_code":401}}
```

Očekivano — API key je expirao. Riot Development keyevi traju samo 24 sata.

Koraci za novi key

1. Idi na developer.riotgames.com
2. Prijavi se s League of Legends računom
3. Na dashboardu klikni "Regenerate" pored Development API Key
4. Kopiraj novi key (izgleda kao RGAPI-xxxxxxx-xxxx-xxxx-xxxx-xxxxxxxxxxxx)

Gdje ga upisat u projektu

Otvori `Services/RiotServices.cs` i zamijeni stari key:

```
private string apiKey = "RGAPI-NOVI-KEY-OVDJE";
```

Spremi, `Ctrl+C`, `dotnet run`, pa opet probaj:

```
http://localhost:5044/PostMatch?matchId=EUW1_7796385639
```

⚠ Svaki put kad pokrećeš projekt za razvoj, provjeri je li key još validan — expira točno 24h nakon generiranja.

Imam dobar key, generiran prije 1h

Onda je problem što key nije ispravno proslijeđen u headeru. Provjeri `RiotServices.cs` — ako si koristio moju verziju s `AddHttpClient<RiotService>()`, problem može biti što se header dodaje više puta.

Ispravi `GetMatchAsync` ovako:

```
public async Task<string> GetMatchAsync(string matchId)
{
    using var request = new HttpRequestMessage(HttpMethod.Get,
        $"https://europe.api.riotgames.com/lol/match/v5/matches/{matchId}");

    request.Headers.Add("X-Riot-Token", apiKey);

    var response = await _httpClient.SendAsync(request);
    return await response.Content.ReadAsStringAsync();
}
```

Alternativno — testiraj key direktno u browseru

Otvori ovaj URL direktno (zamijeni key):

```
https://europe.api.riotgames.com/lol/match/v5/matches/EUW1_7796385639?api_key=RGAPI-TV0J-KEY
```

Riot API podržava i `api_key` kao query parametar — ako ovo radi u browseru a projekt ne, znači da je problem u headeru unutar koda. Ako ni ovo ne radi, key nije valjan.

Što dobiveš kad testiraš direktno u browseru?

a gdje je taj async

U `Services/RiotServices.cs` — zamijeni cijelu `GetMatchAsync` metodu:

```
namespace PostMatchSummary.Services
{
    public class RiotService
    {
        private readonly HttpClient _httpClient;
        private string apiKey = "RGAPI-TV0J-KEY-OVDJE";

        public RiotService(HttpClient httpClient)
        {
            _httpClient = httpClient;
        }

        public async Task<string> GetMatchAsync(string matchId)
        {
            using var request = new HttpRequestMessage(HttpMethod.Get,
                $"https://europe.api.riotgames.com/lol/match/v5/matches/{matchId}");

            request.Headers.Add("X-Riot-Token", apiKey);

            var response = await _httpClient.SendAsync(request);
            return await response.Content.ReadAsStringAsync();
        }
    }
}
```

Spremi → `Ctrl+C` → `dotnet run` → probaj opet URL. Što piše u browseru kad testiraš key direktno?

Match Data

```
{"metadata":{"dataVersion":"2","matchId":"EUW1_7796385639","participants":
["E5HqRAUQiffhAL52ujylD9TAo_rwM8a_JT65osce_c61-
ou68OWNv5RdkmUvd6Bg2AvTMwgb3Wmxvw","oM0e5ckTrND5
```

dobio sam nesto ovako dugacko jako

Radi! ☑ To je raw JSON od Riot API-ja s kompletnim podacima o matchu.

Što sad možeš napraviti

Trenutno view samo ispisuje sirovi JSON kao tekst. Sljedeći logičan korak je **parsati taj JSON u C# modele i prikazati ga lijepo u HTML tablici.**

Trebat ćeš:

1. Deserijalizirati JSON u `RiotResponse` model koristeći `System.Text.Json`
2. Prikazati podatke u Razor viewu — igrači, championsi, KDA...

Želiš da ti napišem kako parsati taj JSON i prikazati ga kao normalnu stranicu s tablicom igrača, ili je za sada dovoljno da radi ovako s raw JSONom?

Hocu ga parsati

Trebamo napraviti 4 stvari: novi modeli za JSON strukturu, update controllera, i novi view.

1. Models/RiotResponse.cs

```
using System.Text.Json.Serialization;

namespace PostMatchSummary.Models
{
    public class RiotResponse
    {
        [JsonPropertyName("metadata")]
        public Metadata Metadata { get; set; } = new();

        [JsonPropertyName("info")]
        public MatchInfo Info { get; set; } = new();
    }

    public class Metadata
    {
        [JsonPropertyName("matchId")]
        public string MatchId { get; set; } = string.Empty;
    }

    public class MatchInfo
    {
        [JsonPropertyName("gameDuration")]
        public int GameDuration { get; set; }

        [JsonPropertyName("gameMode")]
        public string GameMode { get; set; } = string.Empty;

        [JsonPropertyName("participants")]
        public List<Participant> Participants { get; set; } = new();
    }

    public class Participant
    {
        [JsonPropertyName("summonerName")]
        public string SummonerName { get; set; } = string.Empty;

        [JsonPropertyName("riotIdGameName")]
        public string RiotIdGameName { get; set; } = string.Empty;

        [JsonPropertyName("championName")]
        public string ChampionName { get; set; } = string.Empty;

        [JsonPropertyName("kills")]
        public int Kills { get; set; }

        [JsonPropertyName("deaths")]
        public int Deaths { get; set; }

        [JsonPropertyName("assists")]
        public int Assists { get; set; }
    }
}
```

```

        [JsonPropertyName("totalMinionsKilled")]
        public int CS { get; set; }

        [JsonPropertyName("goldEarned")]
        public int GoldEarned { get; set; }

        [JsonPropertyName("win")]
        public bool Win { get; set; }

        [JsonPropertyName("teamId")]
        public int TeamId { get; set; }
    }
}

```

2. Services/RiotServices.cs

```

using System.Text.Json;
using PostMatchSummary.Models;

namespace PostMatchSummary.Services
{
    public class RiotService
    {
        private readonly HttpClient _httpClient;
        private string apiKey = "RGAPI-TV0J-KEY-OVDJE";

        public RiotService(HttpClient httpClient)
        {
            _httpClient = httpClient;
        }

        public async Task<RiotResponse?> GetMatchAsync(string matchId)
        {
            using var request = new HttpRequestMessage(HttpMethod.Get,
                $"https://europe.api.riotgames.com/lol/match/v5/matches/{matchId}");

            request.Headers.Add("X-Riot-Token", apiKey);

            var response = await _httpClient.SendAsync(request);
            var json = await response.Content.ReadAsStringAsync();

            return JsonSerializer.Deserialize<RiotResponse>(json);
        }
    }
}

```

3. Controllers/PostMatchController.cs

```

using Microsoft.AspNetCore.Mvc;
using PostMatchSummary.Services;

namespace PostMatchSummary.Controllers
{
    public class PostMatchController : Controller
    {

```

```

        private readonly RiotService _riotService;

        public PostMatchController(RiotService riotService)
        {
            _riotService = riotService;
        }

        public async Task<IActionResult> Index(string matchId)
        {
            var data = await _riotService.GetMatchAsync(matchId);
            return View(data);
        }
    }
}

```

4. Views/PostMatch/Index.cshtml

```

@model PostMatchSummary.Models.RiotResponse

<h2>Match: @Model.Metadata.MatchId</h2>
<p>Trajanje: @(Model.Info.GameDuration / 60) min | Mode: @Model.Info.GameMode</p>

@foreach (var team in Model.Info.Participants.GroupBy(p => p.TeamId))
{
    var won = team.First().Win;
    <h3>Tim @team.Key – @(won ? "✓ Pobjeda" : "✗ Poraz")</h3>
    <table border="1" cellpadding="6">
        <thead>
            <tr>
                <th>Igrač</th>
                <th>Champion</th>
                <th>K/D/A</th>
                <th>CS</th>
                <th>Gold</th>
            </tr>
        </thead>
        <tbody>
            @foreach (var p in team)
            {
                <tr>
                    <td>@(string.IsNullOrEmpty(p.RiotIdGameName) ? p.SummonerName : p.RiotIdGameName)</td>
                    <td>@p.ChampionName</td>
                    <td>@p.Kills/@p.Deaths/@p.Assists</td>
                    <td>@p.CS</td>
                    <td>@p.GoldEarned</td>
                </tr>
            }
        </tbody>
    </table>
}

```

dotnet run pa idi na:

```
http://localhost:5044/PostMatch?matchId=EUW1_7796385639
```

Trebao bi vidjeti lijepu tablicu s dva tima, igračima i njihovim KDA statistikama.☒

An unhandled exception occurred while processing the request.

InvalidOperationException: The view '{\"metadata\":

{\"dataVersion\":\"2\",\"matchId\":\"EUW1_7796385639\",\"participants\":

[\"E5HqRAUQiffhAL52ujlD9TAo_rwM8a_JT65osce_c61-

ou68OWNv5RdkmUvd6Bg2AvTMwgb3Wmxvw\",\"_oM0e5ckTrND5LcFMnKDDvdyQAeVYEP6o3Iwy1FaA-

dko3Am4KBsq8G7t4kqROXyvOp28lwGxyOv5w\",\"9tAtIHnOkHNUU0q7jXLKvpode2n2eTf2hJNJqCLEWhrd

n1bUdwc9Mb0xuZuPsL_RCEM7aoqaNNArw\",\"2nQO588pbuuLS8XYJfFmk1MX-

ZMhSu_isCyP90IHgPdNufRLZTfTXOKVaw2QsCnTir2J6rj874xZbA\",\"p4MSfH5rXQQ57fnLlN2h3i2jOS5R9IV99

rrkfSBKR-ozq0uvjiGIqY_uYct-

ZmtdHegdbihhsmhrtQ\",\"aRZHKlm8mt1ht6LnjZSRfl3lUgG9FuC9Qs18ge6w9K2UJA71KSR0o4PFCcMg8yA

DvW868VlMm5AZg\",\"R3EYf6JwIZYpCClWNNdQf_uIUbVhGuqXuY1jyJxAg2afB_I5ZQDXKfLpO4LhVdPFG4

SKELIye4EDZA\",\"LxlnlWkIAojxKEDSseE5Lw8Jp1H2EoOSfQ5mJEtvdvdRh5f5829IHVJUHLbNh1vwX1xp6lpR

uofcMlg\",\"OMB92kjbLV7sFuKpYLwSot5h6Zf67YyYL_DN6JO09behmdaBEK_y2Q763lQyiuB6sJrLZWxwrE73

g\",\"9B2bf4Alq1sVV5sh73MUea4IPBHnYXTnI50cY5M9ea7Yxy81sS5rRonvWMvMjuYnYaf5cM8DOZRdXA\"]},i

nfo\":

{\"endOfGameResult\":\"GameComplete\",\"gameCreation\":1774350929484,\"gameDuration\":2352,\"gameEndTime

stamp\":1774353298562,\"gameId\":7796385639,\"gameMode\":\"CLASSIC\",\"gameName\":\"teambuilder-match-

7796385639\",\"gameStartTimestamp\":1774350946674,\"gameType\":\"MATCHED_GAME\",\"gameVersion\":\"16.6.756.9

31\",\"mapId\":11,\"participants\":[{\"PlayerBehavior\":

{\"PlayerBehavior_IsHeroInCombat\":0,\"PlayerScore0\":0,\"PlayerScore1\":0,\"PlayerScore10\":0,\"PlayerScore11\":0,\"

PlayerScore2\":0,\"PlayerScore3\":0,\"PlayerScore4\":0,\"PlayerScore5\":0,\"PlayerScore6\":0,\"PlayerScore7\":0,\"PlayerS

core8\":0,\"PlayerScore9\":0,\"allInPings\":0,\"assistMePings\":0,\"assists\":8,\"baronKills\":0,\"basicPings\":0,\"challenges

\":

{\"12AssistStreakCount\":0,\"HealFromMapSources\":11764481048583984,\"InfernalScalePickup\":0,\"SWARM_De

featAatrox\":0,\"SWARM_DefeatBriar\":0,\"SWARM_DefeatMiniBosses\":0,\"SWARM_EvolveWeapon\":0,\"SWARM

_Have3Passives\":0,\"SWARM_KillEnemy\":0,\"SWARM_PickupGold\":0,\"SWARM_ReachLevel50\":0,\"SWARM_Sur

vive15Min\":0,\"SWARM_WinWith5EvolvedWeapons\":0,\"abilityUses\":464,\"acesBefore15Minutes\":0,\"alliedJung

leMonsterKills\":0,\"baronBuffGoldAdvantageOverThreshold\":1,\"baronTakedowns\":1,\"blastConeOppositeO

pponentCount\":0,\"bountyGold\":0,\"buffsStolen\":0,\"completeSupportQuestInTime\":0,\"controlWardsPlaced\":

0,\"damagePerMinute\":709.234088081006,\"damageTakenOnTeamPercentage\":0.24536141431676436,\"danced

WithRiftHerald\":0,\"deathsByEnemyChamps\":3,\"dodgeSkillShotsSmallWindow\":0,\"doubleAces\":0,\"dragonTa

kedowns\":1,\"earliestBaron\":1726406115538,\"earliestDragonTakedown\":1134.5085056100002,\"earlyLaningPh

aseGoldExpAdvantage\":0,\"effectiveHealAndShielding\":3520.75732421875,\"elderDragonKillsWithOpposing

Soul\":0,\"elderDragonMultikills\":0,\"enemyChampionImmobilizations\":83,\"enemyJungleMonsterKills\":3,\"epic

MonsterKillsNearEnemyJungler\":0,\"epicMonsterKillsWithin30SecondsOfSpawn\":0,\"epicMonsterSteals\":0,\"

epicMonsterStolenWithoutSmite\":0,\"firstTurretKilled\":1,\"firstTurretKilledTime\":880.8083404140001,\"fistBu

mpParticipation\":0,\"flawlessAces\":0,\"fullTeamTakedown\":0,\"gameLength\":2352.226227523,\"getTakedownsIn

AllLanesEarlyJungleAsLaner\":0,\"goldPerMinute\":408.21361078432284,\"hadOpenNexus\":0,\"immobilizeAnd

KillWithAlly\":7,\"initialBuffCount\":0,\"initialCrabCount\":0,\"jungleCsBefore10Minutes\":0,\"junglerTakedowns

NearDamagedEpicMonster\":0,\"kTurretsDestroyedBeforePlatesFall\":0,\"kda\":5,\"killAfterHiddenWithAlly\":1,\"

killParticipation\":0.40540540540540543,\"killedChampTookFullTeamDamageSurvived\":0,\"killingSprees\":1,\"k

illsNearEnemyTurret\":3,\"killsOnOtherLanesEarlyJungleAsLaner\":0,\"killsOnRecentlyHealedByAramPack\":0

,\"killsUnderOwnTurret\":0,\"killsWithHelpFromEpicMonster\":0,\"knockEnemyIntoTeamAndKill\":8,\"landSkill

ShotsEarlyGame\":0,\"laneMinionsFirst10Minutes\":81,\"laningPhaseGoldExpAdvantage\":0,\"legendaryCount

\":0,\"legendaryItemUsed\":

[6662,6665,2504,6664,3075],\"lostAnInhibitor\":0,\"maxCsAdvantageOnLaneOpponent\":2575,\"maxKillDeficit

\":0,\"maxLevelLeadLaneOpponent\":3,\"mejaisFullStackInTime\":0,\"moreEnemyJungleThanOpponent\":0,\"mult

iKillOneSpell\":0,\"multiTurretRiftHeraldCount\":0,\"multikills\":0,\"multikillsAfterAggressiveFlash\":0,\"outerTur

```
retExecutesBefore10Minutes":0,"outnumberedKills":1,"outnumberedNexusKill":0,"perfectDragonSoulsTaken":0,"perfectGame":0,"pickKillWithAlly":10,"playedChampSelectPosition":1,"poroExplosions":0,"quickCleanse":0,"quickFirstTurret":0,"quickSoloKills":0,"riftHeraldTakedowns":0,"saveAllyFromDeath":0,"scuttleCrabKills":0,"skillshotsDodged":77,"skillshotsHit":0,"snowballsHit":0,"soloBaronKills":0,"soloKills":2,"stealthWardsPlaced":13,"survivedSingleDigitHpCount":0,"survivedThreeImmobilizesInFight":17,"takedownOnFirstTurret":0,"takedowns":15,"takedownsAfterGainingLevelAdvantage":0,"takedownsBeforeJungleMinionSpawn":0,"takedownsFirstXMinutes":4,"takedownsInAlcove":0,"takedownsInEnemyFountain":0,"teamBaronKills":1,"teamDamagePercentage":0.17475951767462683,"teamElderDragonKills":0,"teamRiftHeraldKills":1,"tookLargeDamageSurvived":1,"turretPlatesTaken":10,"turretTakedowns":2,"turretsTakenWithRiftHerald":5,"twentyMinionsIn3SecondsCount":0,"twoWardsOneSweeperCount":0,"unseenRecalls":0,"visionScoreAdvantageLaneOpponent":0.025036334991455078,"visionScorePerMinute":1.227348569444985,"voidMonsterKill":1,"wardTakedowns":4,"wardTakedownsBefore20M":0,"wardsGuarded":2,"champExperience":25570,"champLevel":20,"championId":897,"championName":"KSante","championTransform":0,"commandPings":7,"consumablesPurchased":1,"damageDealtToBuildings":8119,"damageDealtToEpicMonsters":2723,"damageDealtToObjectives":10843,"damageDealtToTurrets":8119,"damageSelfMitigated":117176,"dangerPings":0,"deaths":3,"detectorWardsPlaced":0,"doubleKills":0,"dragonKills":0,"eligibleForProgression":true,"enemyMissingPings":2,"enemyVisionPings":0,"firstBloodAssist":false,"firstBloodKill":false,"firstTowerAssist":false,"firstTowerKill":false,"gameEndedInEarlySurrender":false,"gameEndedInSurrender":false,"getBackPings":0,"goldEarned":16003,"goldSpent":16100,"holdPings":0,"individualPosition":"TOP","inhibitorKills":0,"inhibitorTakedowns":0,"inhibitorsLost":3,"item0":6665,"item1":2504,"item2":3047,"item3":6662,"item4":3075,"item5":6664,"item6":3340,"itemsPurchased":21,"killingSprees":1,"kills":7,"lane":"JUNGLE","largestCriticalStrike":6,"largestKillingSpree":7,"largestMultiKill":1,"longestTimeSpentLiving":1997,"magicDamageDealt":16353,"magicDamageDealtToChampions":2554,"magicDamageTaken":15228,"missions":{},"playerScore0":0,"playerScore1":0,"playerScore2":0,"playerScore3":0,"playerScore4":0,"playerScore5":0,"playerScore6":0,"playerScore7":0,"playerScore8":0}
```

nesto dugo

Greška je jasna — `RiotServices.cs` još uvijek vraća string umjesto `RiotResponse`, pa controller šalje JSON string kao naziv viewa!

Provjeri je li `RiotServices.cs` ažuriran s mojom verzijom. Metoda mora vraćati `Task<RiotResponse?>`, ne `Task<string>`:

Services/RiotServices.cs — mora biti OVAKO:

```
using System.Text.Json;
using PostMatchSummary.Models;

namespace PostMatchSummary.Services
{
    public class RiotService
    {
        private readonly HttpClient _httpClient;
        private string apiKey = "RGAPI-TVOJ-KEY-OVDJE";

        public RiotService(HttpClient httpClient)
        {
            _httpClient = httpClient;
        }

        public async Task<RiotResponse?> GetMatchAsync(string matchId) // ← RiotResponse? ne string
        {
            using var request = new HttpRequestMessage(HttpMethod.Get,
```

```

        $"https://europe.api.riotgames.com/lol/match/v5/matches/{matchId}");

        request.Headers.Add("X-Riot-Token", apiKey);

        var response = await _httpClient.SendAsync(request);
        var json = await response.Content.ReadAsStringAsync();

        return JsonSerializer.Deserialize<RiotResponse>(json); // ← parsamo JSON!
    }
}

```

Controllers/PostMatchController.cs — mora biti OVAKO:

```

using Microsoft.AspNetCore.Mvc;
using PostMatchSummary.Services;

namespace PostMatchSummary.Controllers
{
    public class PostMatchController : Controller
    {
        private readonly RiotService _riotService;

        public PostMatchController(RiotService riotService)
        {
            _riotService = riotService;
        }

        public async Task<IActionResult> Index(string matchId)
        {
            var data = await _riotService.GetMatchAsync(matchId);
            return View(data); // ← bez "Index" kao prvi parametar!
        }
    }
}

```

Ctrl+C → dotnet run → probaj ponovo. ☒

PostMatchSummary

[Home](#)

[Privacy](#)

Match: EUW1_7796385639

Trajanje: 39 min | Mode: CLASSIC

Tim 100 — ✖ Poraz

Igrač Champion K/D/ACSGold

Liuval

KSante

7/3/8

324

16003

bamses ven

Kayn

8/3/12

42
17772
Seijun
Mel
16/9/6
292
20306
FsG kasterZ
Velkoz
5/13/7
230
13745
do not disturb
Amumu
1/12/14
31
10193
Tim 200 — ✔ Pobjeda
IgračChampionK/D/ACSGold
MZP Westman
Gnar
3/7/6
327
16931
provinko
LeeSin
11/7/14
39
16785
PatatoMajor
Ahri
5/9/14
291
17381
asol 아우렐리온 솔
AurelionSol
19/5/13
279
21125
Tisaur
Leona
1/9/26
33
11493

dobijem ovakav prikaz

Savršeno radi! ☑ Imaš kompletan post-match summary s oba tima, igračima, championima i KDA statistikama.

Sad možeš uljepšati prikaz ako želiš — evo nekoliko ideja što možeš dodati:

Moguća poboljšanja

- CSS stilizacija — bojanje redova po timu (plavo/crveno), highlight pobjedničkog tima
- KDA ratio — izračunati (Kills + Assists) / Deaths i prikazati
- Forma za unos — input polje na stranici umjesto da ručno pišeš matchId u URL
- Lista više matcheva — dohvatiti zadnjih N matcheva nekog igrača po PUUID-u
- Champion ikone — dohvatiti slike iz Riot Data Dragon CDN-a

Što te najviše zanima kao sljedeći korak?

okej sada u kodu, bio je patch igrice se updateala, sada kada stavim novi API, jer onaj je expireo, mi stranica bude prazna tj ne izbaci mi ni json ni lijepo uređeno tablicu kako smo slozili

Najvjerojatnije je problem što se JSON struktura promijenila u novom patchu, pa deserijalizacija faili tiho i vraća null.

Dijagnostika — provjeri što API vraća

Privremeno u `PostMatchController.cs` dodaj fallback da vidiš raw JSON:

```
public async Task<IActionResult> Index(string matchId)
{
    var data = await _riotService.GetMatchAsync(matchId);

    if (data == null)
        return Content("Deserijalizacija failala - provjeri JSON strukturu");

    return View(data);
}
```

Ako piše "Deserijalizacija failala", problem je u modelu.

Brzi fix — dodaj raw JSON metodu u RiotServices.cs

Privremeno dodaj metodu koja vraća string da vidiš što Riot API šalje:

```
public async Task<string> GetRawJsonAsync(string matchId)
{
    using var request = new HttpRequestMessage(HttpMethod.Get,
        $"https://europe.api.riotgames.com/lol/match/v5/matches/{matchId}");

    request.Headers.Add("X-Riot-Token", apiKey);

    var response = await _httpClient.SendAsync(request);
    return await response.Content.ReadAsStringAsync();
}
```

Pozovije u controlleru i vrati `Content(rawJson)` — pa paste-aj ovdje što dobiješ, da vidim je li se struktura promijenila i popravim modele. ☒

Services/RiotServices.cs

```
using System.Text.Json;
using PostMatchSummary.Models;

namespace PostMatchSummary.Services
{
    public class RiotService
    {
        private readonly HttpClient _httpClient;
        private string apiKey = "RGAPI-TVOJ-KEY-OVDJE";

        public RiotService(HttpClient httpClient)
        {
            _httpClient = httpClient;
        }

        public async Task<RiotResponse?> GetMatchAsync(string matchId)
        {
            using var request = new HttpRequestMessage(HttpMethod.Get,
                $"https://europe.api.riotgames.com/lol/match/v5/matches/{matchId}");

            request.Headers.Add("X-Riot-Token", apiKey);

            var response = await _httpClient.SendAsync(request);
            var json = await response.Content.ReadAsStringAsync();

            return JsonSerializer.Deserialize<RiotResponse>(json);
        }

        // ← PRIVREMENA METODA ZA DIJAGNOSTIKU
        public async Task<string> GetRawJsonAsync(string matchId)
        {
            using var request = new HttpRequestMessage(HttpMethod.Get,
                $"https://europe.api.riotgames.com/lol/match/v5/matches/{matchId}");

            request.Headers.Add("X-Riot-Token", apiKey);

            var response = await _httpClient.SendAsync(request);
            return await response.Content.ReadAsStringAsync();
        }
    }
}
```

Controllers/PostMatchController.cs

```
using Microsoft.AspNetCore.Mvc;
using PostMatchSummary.Services;

namespace PostMatchSummary.Controllers
{
    public class PostMatchController : Controller
    {
        private readonly RiotService _riotService;

        public PostMatchController(RiotService riotService)
        {
            _riotService = riotService;
        }
    }
}
```

```

    }

    public async Task<IActionResult> Index(string matchId)
    {
        // DIJAGNOSTIKA - privremeno
        var rawJson = await _riotService.GetRawJsonAsync(matchId);
        return Content(rawJson, "application/json");

        // ← OVO JE ZAKOMENTIRANO DOK NE POPRAVIMO MODELE
        // var data = await _riotService.GetMatchAsync(matchId);
        // if (data == null)
        //     return Content("Deserijalizacija failala");
        // return View(data);
    }
}

```

Ctrl+C → dotnet run → otvori:

```
http://localhost:5044/PostMatch?matchId=EUW1_NOVI_MATCH_ID
```

Paste-aj što dobiješ (ili prvi dio JSONa) pa ću popraviti modele. ☒

Kada saveam kod i runnam link samo se ovako prikaze

Vidiš Match: i Trajanje: 0 min | Mode: — to znači da controller **nije promijenjen** i još uvijek poziva GetMatchAsync (koji vraća prazan model), a ne dijagnostičku GetRawJsonAsync. ^[59]

Provjeri jesi li stvarno snimio PostMatchController.cs s novim kodom. Controller mora izgledati točno ovako:

```

using Microsoft.AspNetCore.Mvc;
using PostMatchSummary.Services;

namespace PostMatchSummary.Controllers
{
    public class PostMatchController : Controller
    {
        private readonly RiotService _riotService;

        public PostMatchController(RiotService riotService)
        {
            _riotService = riotService;
        }

        public async Task<IActionResult> Index(string matchId)
        {
            // DIJAGNOSTIKA
            var rawJson = await _riotService.GetRawJsonAsync(matchId);
            return Content(rawJson, "application/json");
        }
    }
}

```

Također provjeri URL — mora imati matchId parametar:

```
http://localhost:5044/PostMatch?matchId=EUW1_7796385639
```

Bez ?matchId=. . . controller dobiva null i vraća prazan prikaz. Što piše u browseru nakon ove promjene?



Yup, nula bodova

Znači controller prima null za matchId. Provjeri URL — moraš imati:

```
http://localhost:5044/PostMatch?matchId=EUW1_7796385639
```

Brzi test — hardkodiraj matchId privremeno

U controlleru privremeno hardkodiraj matchId da eliminiramo problem s URL parametrom:

```
public async Task<IActionResult> Index(string matchId)
{
    // PRIVREMENO - hardkodirani matchId
    var testId = "EUW1_7796385639";
    var rawJson = await _riotService.GetRawJsonAsync(testId);
    return Content(rawJson, "application/json");
}
```

Sad idi na:

```
http://localhost:5044/PostMatch
```

Bez ikakvih parametara. Što dobiješ?

- Ako dobiješ JSON → problem je bio u URL parametru
- Ako dobiješ prazno/grešku → problem je u API keyu ili RiotServices.cs
- Ako dobiješ {"status":{"message":"Forbidden"}} → API key je expirao

I dalje prazno

Znači problem je u RiotServices.cs ili API keyu. Dodaj error handling da vidimo točno što se događa:

Controllers/PostMatchController.cs

```
using Microsoft.AspNetCore.Mvc;
using PostMatchSummary.Services;

namespace PostMatchSummary.Controllers
{
    public class PostMatchController : Controller
    {
        private readonly RiotService _riotService;

        public PostMatchController(RiotService riotService)
        {

```

```

        _riotService = riotService;
    }

    public async Task<IActionResult> Index(string matchId)
    {
        try
        {
            var testId = "EUW1_7796385639";
            var rawJson = await _riotService.GetRawJsonAsync(testId);
            return Content("ODGOVOR: " + rawJson, "text/plain");
        }
        catch (Exception ex)
        {
            return Content("GREŠKA: " + ex.Message + "\n\n" + ex.StackTrace, "text/plain");
        }
    }
}

```

Ctrl+C → dotnet run → otvori <http://localhost:5044/PostMatch>

Sada mora nešto pisati — ili JSON, ili konkretnu grešku. Što piše?

using System.Text.Json;

```
using PostMatchSummary.Models;
```

```
namespace PostMatchSummary.Services
```

```

{
    public class RiotService
    {
        private readonly HttpClient _httpClient;
        private string apiKey = "RGAPI-TVOJ-KEY-OVDJE";

        public RiotService(HttpClient httpClient)
        {
            _httpClient = httpClient;
        }

        public async Task<RiotResponse?> GetMatchAsync(string matchId)
        {
            using var request = new HttpRequestMessage(HttpMethod.Get,
                $"https://europe.api.riotgames.com/lol/match/v5/matches/{matchId}");

            request.Headers.Add("X-Riot-Token", apiKey);

            var response = await _httpClient.SendAsync(request);
            var json = await response.Content.ReadAsStringAsync();

            return JsonSerializer.Deserialize<RiotResponse>(json);
        }

        // ← PRIVREMENA METODA ZA DIJAGNOSTIKU
        public async Task<string> GetRawJsonAsync(string matchId)
        {

```

```

using var request = new HttpRequestMessage(HttpMethod.Get,
    $"https://europe.api.riotgames.com/lol/match/v5/matches/{matchId}");

request.Headers.Add("X-Riot-Token", apiKey);

var response = await _httpClient.SendAsync(request);
return await response.Content.ReadAsStringAsync();
}
}
}

```

ovo mi je trenutni

Vidim problem odmah — nisi promijenio API key!

```
private string apiKey = "RGAPI-TVOJ-KEY-OVDJE"; // ← OVO JE PLACEHOLDER!
```

Idi na developer.riotgames.com, kopiraj svoj trenutni key i zamijeni:

```
private string apiKey = "RGAPI-xxxx-tvoj-pravi-key";
```

Ctrl+C → dotnet run → http://localhost:5044/PostMatch

To je bio cijelo vrijeme problem. 😊

C:\Users\Matej\Desktop\ASP.NET \PostMatchSummary>dotnet run

Using launch settings from C:\Users\Matej\Desktop\ASP.NET
\PostMatchSummary\Properties\launchSettings.json...

Building...

C:\Users\Matej\Desktop\ASP.NET\PostMatchSummary\Models\Champion.cs(3,19): warning CS8618: Non-nullable property 'Name' must contain a non-null value when exiting constructor. Consider adding the 'required' modifier or declaring the property as nullable.

info: Microsoft.Hosting.Lifetime[14]

Now listening on: <http://localhost:5044>

info: Microsoft.Hosting.Lifetime[0]

Application started. Press Ctrl+C to shut down.

info: Microsoft.Hosting.Lifetime[0]

Hosting environment: Development

info: Microsoft.Hosting.Lifetime[0]

Content root path: C:\Users\Matej\Desktop\ASP.NET\PostMatchSummary

JSON radi! Struktura je ista kao prije, znači modeli su ispravni. Problem je samo što controller još uvijek ima dijagnostički kod. [\[60\]](#)

Sad vrati controller na normalan prikaz tablice:

Controllers/PostMatchController.cs

```
using Microsoft.AspNetCore.Mvc;
using PostMatchSummary.Services;

namespace PostMatchSummary.Controllers
{
    public class PostMatchController : Controller
    {
        private readonly RiotService _riotService;

        public PostMatchController(RiotService riotService)
        {
            _riotService = riotService;
        }

        public async Task<IActionResult> Index(string matchId)
        {
            var data = await _riotService.GetMatchAsync(matchId);

            if (data == null)
                return Content("Deserijalizacija failala");

            return View(data);
        }
    }
}
```

Ctrl+C → dotnet run → Otvori:

http://localhost:5044/PostMatch?matchId=EUW1_7796654199

Sada bi trebala raditi lijepa tablica kao i prije! ☒

✱✱

i dalje metadata prikazt

Znači data == null — deserijalizacija faila. Problem je što RiotResponse model možda ne odgovara JSON strukturi.

Dodaj ovo privremeno da vidimo točnu grešku:

Controllers/PostMatchController.cs

```
using Microsoft.AspNetCore.Mvc;
using PostMatchSummary.Services;
using System.Text.Json;
using PostMatchSummary.Models;

namespace PostMatchSummary.Controllers
{
    public class PostMatchController : Controller
    {
        private readonly RiotService _riotService;
```

```

    public PostMatchController(RiotService riotService)
    {
        _riotService = riotService;
    }

    public async Task<IActionResult> Index(string matchId)
    {
        try
        {
            var rawJson = await _riotService.GetRawJsonAsync(matchId ?? "EUW1_7796654199");

            var options = new JsonSerializerOptions
            {
                PropertyNameCaseInsensitive = true
            };

            var data = JsonSerializer.Deserialize<RiotResponse>(rawJson, options);

            if (data == null)
                return Content("data je null nakon deserijalizacije");

            if (data.Info == null)
                return Content("data.Info je null - problem s Info modelom");

            if (data.Info.Participants == null || data.Info.Participants.Count == 0)
                return Content("Participants su prazni");

            return View(data);
        }
        catch (Exception ex)
        {
            return Content("GREŠKA: " + ex.Message);
        }
    }
}

```

Ctrl+C → dotnet run → što piše?

Models/Champion.cs

```

namespace PostMatchSummary.Models
{
    public class Champion
    {
        public string Name { get; set; } = string.Empty;
    }
}

```

Controllers/PostMatchController.cs

```

using Microsoft.AspNetCore.Mvc;
using PostMatchSummary.Services;
using System.Text.Json;
using PostMatchSummary.Models;

```

```

namespace PostMatchSummary.Controllers
{
    public class PostMatchController : Controller
    {
        private readonly RiotService _riotService;

        public PostMatchController(RiotService riotService)
        {
            _riotService = riotService;
        }

        public async Task<IActionResult> Index(string matchId)
        {
            var rawJson = await _riotService.GetRawJsonAsync(matchId ?? "EUW1_7796654199");

            var options = new JsonSerializerOptions
            {
                PropertyNameCaseInsensitive = true
            };

            var data = JsonSerializer.Deserialize<RiotResponse>(rawJson, options);

            if (data == null)
                return Content("Deserijalizacija failala");

            return View(data);
        }
    }
}

```

Ctrl+C → dotnet run → otvori:

```
http://localhost:5044/PostMatch?matchId=EUW1_7796654199
```

Ovo bi sad trebalo raditi — `PropertyNameCaseInsensitive = true` rješava problem što Riot API vraća camelCase JSON ("gameDuration") a naši modeli imaju PascalCase propertyje (GameDuration).☒

odlično, sada imam prikaz, idemo sada kroz sav

also kako da točno vidim što je u riot api-u da popravim neke stvari u tablici kao npr, da piše "blue team" "red team", da CS nije krivi jer ne counta neutral monstere i tako

Odlično, sve radi! Evo odgovora na oba pitanja:^[61]

Kako vidjeti sve Riot API fieldove

Najlakši način — otvori raw JSON koji već dobivaš u browseru i pretraži ga. Privremeno dodaj dijagnostičku rutu u controller pa paste-aj u jsoncrack.com ili jsonformatter.org za vizualni prikaz svih fieldova.

Ispravci u modelu i viewu

1. Blue/Red Team umjesto Tim 100/200

teamId = 100 → Blue side, teamId = 200 → Red side. U Views/PostMatch/Index.cshtml promijeni: ^[62]

```
@{
    var teamName = team.Key == 100 ? "🔵 Blue Team" : "🔴 Red Team";
}
<h3>@teamName – @(won ? "✔ Pobjeda" : "✖ Poraz")</h3>
```

2. Ispravan CS (minioni + neutralni monstri)

totalMinionsKilled broji samo lane minione. Za pravi CS treba dodati neutralMinionsKilled. U Models/RiotResponse.cs dodaj property: ^[63]

```
[JsonPropertyName("neutralMinionsKilled")]
public int NeutralMinionsKilled { get; set; }
```

Pa u viewu promijeni prikaz CS-a:

```
<td>@(p.CS + p.NeutralMinionsKilled)</td>
```

3. Bonus fieldovi koje možeš dodati u model

JSON field	Što je
individualPosition	TOP, JUNGLE, MID, BOTTOM, UTILITY
visionScore	Vision score igrača
totalDamageDealtToChampions	Damage na championa
wardsPlaced	Wardi postavljeni
doubleKills, tripleKills, quadraKills, pentaKills	Multikillovi

Dodaješ ih u Participant klasu u RiotResponse.cs s [JsonPropertyName("fieldName")] atributom, a onda ih koristiš u viewu.



dal ja uopce koristim ista od "modela" iz svog koda ili sve ide api pa prikaz

Dobro pitanje! Gotovo ništa od tvojih originalnih modela se ne koristi.

Što se zapravo koristi

Jedino što se koristi je RiotResponse.cs koji smo napisali zajedno — ali to je **novi model**, ne tvoji originalni.

Tvoji originalni modeli — svi su beskorisni

File	Status	Razlog
Match.cs	✗ Ne koristi se	API vraća RiotResponse, ne Match
Player.cs	✗ Ne koristi se	API vraća Participant, ne Player
Team.cs	✗ Ne koristi se	Timovi se grupiraju po teamId iz Participant
MatchSummary.cs	✗ Ne koristi se	Nigdje se ne poziva
Champion.cs	✗ Ne koristi se	Champion je samo string u Participant
ChampionRole.cs	✗ Ne koristi se	Nigdje se ne poziva
RiotResponse.cs (stari)	✗ Bio prazan	Zamijenili smo ga novim

Tok podataka u tvom projektu

```
Riot API (JSON)
↓
RiotServices.cs → deserijalizira u RiotResponse.cs (novi)
↓
PostMatchController.cs → šalje model u View
↓
Views/PostMatch/Index.cshtml → prikazuje tablicu
```

Što napraviti

Možeš **pobrisati** ove fileove iz Models/ foldera jer samo zauzimaju prostor:

- Match.cs
- Player.cs
- Team.cs
- MatchSummary.cs
- Champion.cs
- ChampionRole.cs

Ostavljaj samo RiotResponse.cs i ErrorViewModel.cs.

Jel ja zadovoljavam ovo onda, jer ako se nista od mojih klasa ne koristi

Pročitao sam lab upute. Imaš problem — zadatak zahtijeva specifične stvari koje trenutno nisu ispunjene. [\[77\]](#)

Što zadatak traži vs. što imaš

Zahtjev	Status	Problem
Barem 7 klasa	△ Djelomično	Imaš klase ali nisu dobro strukturirane
4 klase s 5+ svojstava	✗ Ne	Tvoje klase imaju 2-4 svojstva

Zahtjev	Status	Problem
Barem jedan enum	✗ Ne	ChampionRole.cs je enum ali nije korišten
Barem jedno DateTime	⚠	Match.cs ima GameCreation ali se ne koristi
Ispravne konekcije 1-N, N-N	✗ Ne	Nema pravih veza između klasa
Napuniti objekte s 3 glavna objekta	✗ Ne	Main.cs je obrisao
LINQ upiti	✗ Ne	Obrisani zajedno s Main.cs

Što treba napraviti

Zadatak traži odvojeni objektni model koji se posebno "puni" podacima i ima LINQ upite — to je bio Main.cs i Program.cs u Models/ folderu koji si obrisao! Taj kod je bio točno ono što se tražilo, samo loše organiziran.

Trebaš:

1. Popraviti modele — dodati više propertyja, enum, DateTime, veze između klasa
2. Rekreatirati Main.cs u Models/ folderu s podacima i LINQ upitima
3. Pushati na GitHub do sutra (3.4.2026.)

Želiš da ti napišem kompletan ispravan objektni model s popunjenim podacima i LINQ upitima koji zadovoljava sve zahtjeve?



Poslati cu ti sav kod koji imam iz svake klase koje imam, zelim da se na API poziv podatci spremaju u moje klase pod moje podatke i na taj nacin se prikazuju a ne da uzimam riot api->json->prikaz,

```
RiotServices.cs
"using System.Text.Json;
using PostMatchSummary.Models;

namespace PostMatchSummary.Services
{
    public class RiotService
    {
        private readonly HttpClient _httpClient;
        private string apiKey = "RGAPI-1e8c1ce6-3c59-4f6f-a086-dda3b60e09f1";

        public RiotService(HttpClient httpClient)
        {
            _httpClient = httpClient;
        }

        public async Task<RiotResponse?> GetMatchAsync(string matchId)
        {
            using var request = new HttpRequestMessage(HttpMethod.Get,
                $"https://europe.api.riotgames.com/lol/match/v5/matches/{matchId}");
```

```

request.Headers.Add("X-Riot-Token", apiKey);

var response = await _httpClient.SendAsync(request);
var json = await response.Content.ReadAsStringAsync();

return JsonSerializer.Deserialize<RiotResponse>(json);
}

// ← PRIVREMENA METODA ZA DIJAGNOSTIKU
public async Task<string> GetRawJsonAsync(string matchId)
{
    using var request = new HttpRequestMessage(HttpMethod.Get,
        $"https://europe.api.riotgames.com/lol/match/v5/matches/{matchId}");

    request.Headers.Add("X-Riot-Token", apiKey);

    var response = await _httpClient.SendAsync(request);
    return await response.Content.ReadAsStringAsync();
}
}
}"

```

PostMatchController.cs

```

"using Microsoft.AspNetCore.Mvc;
using PostMatchSummary.Services;
using System.Text.Json;
using PostMatchSummary.Models;

namespace PostMatchSummary.Controllers
{
    public class PostMatchController : Controller
    {
        private readonly RiotService _riotService;

        public PostMatchController(RiotService riotService)
        {
            _riotService = riotService;
        }

        public async Task<ActionResult> Index(string matchId)
        {
            var rawJson = await _riotService.GetRawJsonAsync(matchId ?? "EUW1_7796654199");

            var options = new JsonSerializerOptions
            {
                PropertyNameCaseInsensitive = true
            };

            var data = JsonSerializer.Deserialize<RiotResponse>(rawJson, options);

            if (data == null)
                return Content("Deserijalizacija failala");
        }
    }
}

```

```
return View(data);  
    }  
}  
}"
```

Champion.cs

```
"namespace PostMatchSummary.Models  
{  
    public class Champion  
    {  
        public string Name {get;set;} = string.Empty;  
    }  
}"
```

ChampionRole.cs

```
"public enum ChampionRole  
{  
    Top,  
    Jungle,  
    Mid,  
    ADC,  
    Support  
}"
```

Match.cs

```
"namespace PostMatchSummary.Models  
{  
    public class Match  
    {  
        public string MatchId {get;set;} = string.Empty;  
        public DateTime GameCreation {get;set;}  
        public int Duration {get;set;}  
        public List<Player> Players {get;set;} = new();  
        public List<Team> Teams {get;set;} = new();  
    }  
}"
```

MatchSummary.cs

```
"namespace PostMatchSummary.Models  
{  
    public class MatchSummary  
    {  
        public string MatchId {get;set;} = string.Empty;  
        public string WinnerTeam {get;set;} = string.Empty;  
        public int TotalKills {get;set;}  
    }  
}"
```

Player.cs

```
"namespace PostMatchSummary.Models  
{  
    public class Player
```

```

{
    public string SummonerName { get; set; } = string.Empty;
    public string Champion { get; set; } = string.Empty;
    public int Kills { get; set; }
    public int Deaths { get; set; }
    public int Assists { get; set; }
    public Team Team { get; set; } = new();
}
}"

```

RiotResponses.cs

```
"using System.Text.Json.Serialization;
```

```
namespace PostMatchSummary.Models
```

```

{
    public class RiotResponse
    {
        [JsonPropertyName("metadata")]
        public Metadata Metadata { get; set; } = new();

```

```

        [JsonPropertyName("info")]
        public MatchInfo Info { get; set; } = new();
    }

```

```

public class Metadata
{
    [JsonPropertyName("matchId")]
    public string MatchId { get; set; } = string.Empty;
}

```

```

public class MatchInfo
{
    [JsonPropertyName("gameDuration")]
    public int GameDuration { get; set; }

```

```

        [JsonPropertyName("gameMode")]
        public string GameMode { get; set; } = string.Empty;

```

```

        [JsonPropertyName("participants")]
        public List<Participant> Participants { get; set; } = new();
    }

```

```

public class Participant
{
    [JsonPropertyName("summonerName")]
    public string SummonerName { get; set; } = string.Empty;

    [JsonPropertyName("riotIdGameName")]
    public string RiotIdGameName { get; set; } = string.Empty;

    [JsonPropertyName("championName")]
    public string ChampionName { get; set; } = string.Empty;

```

```

[JsonPropertyName("kills")]
    public int Kills {get;set;}

[JsonPropertyName("deaths")]
    public int Deaths {get;set;}

[JsonPropertyName("assists")]
    public int Assists {get;set;}

[JsonPropertyName("totalMinionsKilled")]
    public int CS {get;set;}

[JsonPropertyName("goldEarned")]
    public int GoldEarned {get;set;}

[JsonPropertyName("win")]
    public bool Win {get;set;}

[JsonPropertyName("teamId")]
    public int TeamId {get;set;}
}
}

```

Team.cs

```

namespace PostMatchSummary.Models
{
    public class Team
    {
        public int TeamId {get;set;}
        public bool Win {get;set;}
        public List<Player> Players {get;set;} = new();
    }
}

```

Index.cshtml

```
@model PostMatchSummary.Models.RiotResponse
```

Match: @Model.Metadata.MatchId

Trajanje: @(Model.Info.GameDuration / 60) min | Mode: @Model.Info.GameMode

```

@foreach (var team in Model.Info.Participants.GroupBy(p => p.TeamId))
{
    var won = team.First().Win;

```

Tim @team.Key — @(won ? "✓ Pobjeda" : "✗ Poraz")

```
@foreach (var p in team)
```

```
{
```

```
}
```

Igrač	Champion	K/D/A	CS	Gold
@(string.IsNullOrEmpty(p.RiotIdGameName) ? p.SummonerName : p.RiotIdGameName)	@p.ChampionName	@p.Kills/@p.Deaths/@p.Assists	@p.CS	@p.GoldEarned

```
}"
```

zelim da moji podatci ispunjavaju sve uvjete iz md filea, da se podatci vuku iz API, i spremaju u moj MVC layer te ih nakon spremanje kod sebe prikazujem

Razumijem — želiš da API podaci prolaze kroz tvoje klase (Match, Player, Team, Champion) a ne direktno iz RiotResponse u view. Evo kompletnog rješenja koje zadovoljava sve uvjete iz lab zadatka.

1. Models/ChampionRole.cs

```
namespace PostMatchSummary.Models
{
    public enum ChampionRole
    {
        Top,
        Jungle,
        Mid,
        ADC,
        Support,
        Unknown
    }
}
```

2. Models/Champion.cs

```
namespace PostMatchSummary.Models
{
    public class Champion
    {
        public string Name { get; set; } = string.Empty;
        public ChampionRole Role { get; set; }
        public int Kills { get; set; }
        public int Deaths { get; set; }
        public int Assists { get; set; }
        public int CS { get; set; }
        public int GoldEarned { get; set; }
        public string SummonerName { get; set; } = string.Empty;
    }
}
```

3. Models/Player.cs

```
namespace PostMatchSummary.Models
{
    public class Player
    {
        public string SummonerName { get; set; } = string.Empty;
        public Champion Champion { get; set; } = new();
        public int Kills { get; set; }
        public int Deaths { get; set; }
        public int Assists { get; set; }
        public int CS { get; set; }
        public int GoldEarned { get; set; }
        public bool Win { get; set; }
        public ChampionRole Role { get; set; }
        public Team? Team { get; set; }
    }
}
```

4. Models/Team.cs

```
namespace PostMatchSummary.Models
{
    public class Team
    {
        public int TeamId { get; set; }
        public string TeamName => TeamId == 100 ? "Blue Team" : "Red Team";
        public bool Win { get; set; }
        public List<Player> Players { get; set; } = new();

        public int TotalKills => Players.Sum(p => p.Kills);
        public int TotalGold => Players.Sum(p => p.GoldEarned);
    }
}
```

5. Models/Match.cs

```
namespace PostMatchSummary.Models
{
    public class Match
    {
        public string MatchId { get; set; } = string.Empty;
        public DateTime GameCreation { get; set; }
        public int Duration { get; set; }
        public string GameMode { get; set; } = string.Empty;
        public string GameVersion { get; set; } = string.Empty;
        public List<Player> Players { get; set; } = new();
        public List<Team> Teams { get; set; } = new();

        public Team? WinnerTeam => Teams.FirstOrDefault(t => t.Win);
        public int TotalKills => Players.Sum(p => p.Kills);
    }
}
```

6. Models/MatchSummary.cs

```
namespace PostMatchSummary.Models
{
    public class MatchSummary
    {
        public string MatchId { get; set; } = string.Empty;
        public string WinnerTeamName { get; set; } = string.Empty;
        public int TotalKills { get; set; }
        public int DurationMinutes { get; set; }
        public string GameMode { get; set; } = string.Empty;
        public Player? MVP => null; // može se proširiti
    }
}
```

7. Models/RiotResponse.cs

```
using System.Text.Json.Serialization;

namespace PostMatchSummary.Models
{
    public class RiotResponse
    {
        [JsonPropertyName("metadata")]
        public Metadata Metadata { get; set; } = new();

        [JsonPropertyName("info")]
        public MatchInfo Info { get; set; } = new();
    }

    public class Metadata
    {
        [JsonPropertyName("matchId")]
        public string MatchId { get; set; } = string.Empty;
    }
}
```

```

public class MatchInfo
{
    [JsonPropertyName("gameDuration")]
    public int GameDuration { get; set; }

    [JsonPropertyName("gameMode")]
    public string GameMode { get; set; } = string.Empty;

    [JsonPropertyName("gameVersion")]
    public string GameVersion { get; set; } = string.Empty;

    [JsonPropertyName("gameCreation")]
    public long GameCreation { get; set; }

    [JsonPropertyName("participants")]
    public List<Participant> Participants { get; set; } = new();
}

public class Participant
{
    [JsonPropertyName("riotIdGameName")]
    public string RiotIdGameName { get; set; } = string.Empty;

    [JsonPropertyName("summonerName")]
    public string SummonerName { get; set; } = string.Empty;

    [JsonPropertyName("championName")]
    public string ChampionName { get; set; } = string.Empty;

    [JsonPropertyName("individualPosition")]
    public string IndividualPosition { get; set; } = string.Empty;

    [JsonPropertyName("kills")]
    public int Kills { get; set; }

    [JsonPropertyName("deaths")]
    public int Deaths { get; set; }

    [JsonPropertyName("assists")]
    public int Assists { get; set; }

    [JsonPropertyName("totalMinionsKilled")]
    public int TotalMinionsKilled { get; set; }

    [JsonPropertyName("neutralMinionsKilled")]
    public int NeutralMinionsKilled { get; set; }

    [JsonPropertyName("goldEarned")]
    public int GoldEarned { get; set; }

    [JsonPropertyName("win")]
    public bool Win { get; set; }

    [JsonPropertyName("teamId")]
    public int TeamId { get; set; }
}
}

```

8. Services/RiotServices.cs

```
using System.Text.Json;
using PostMatchSummary.Models;

namespace PostMatchSummary.Services
{
    public class RiotService
    {
        private readonly HttpClient _httpClient;
        private string apiKey = "RGAPI-TV0J-KEY";

        public RiotService(HttpClient httpClient)
        {
            _httpClient = httpClient;
        }

        public async Task<Match?> GetMatchAsync(string matchId)
        {
            using var request = new HttpRequestMessage(HttpMethod.Get,
                $"https://europe.api.riotgames.com/lol/match/v5/matches/{matchId}");

            request.Headers.Add("X-Riot-Token", apiKey);

            var response = await _httpClient.SendAsync(request);
            var json = await response.Content.ReadAsStringAsync();

            var options = new JsonSerializerOptions { PropertyNameCaseInsensitive = true };
            var riotResponse = JsonSerializer.Deserialize<RiotResponse>(json, options);

            if (riotResponse == null) return null;

            // Mapiranje RiotResponse → naše klase
            return MapToMatch(riotResponse);
        }

        private Match MapToMatch(RiotResponse riotResponse)
        {
            var match = new Match
            {
                MatchId = riotResponse.Metadata.MatchId,
                Duration = riotResponse.Info.GameDuration,
                GameMode = riotResponse.Info.GameMode,
                GameVersion = riotResponse.Info.GameVersion,
                GameCreation = DateTimeOffset.FromUnixTimeMilliseconds(riotResponse.Info.GameCre
            };

            // Kreiraj timove
            var team100 = new Team { TeamId = 100 };
            var team200 = new Team { TeamId = 200 };

            foreach (var p in riotResponse.Info.Participants)
            {
                var champion = new Champion
                {
                    Name = p.ChampionName,
                    Role = ParseRole(p.IndividualPosition),
                    Kills = p.Kills,
                    Deaths = p.Deaths,
                    Assists = p.Assists,
                    CS = p.TotalMinionsKilled + p.NeutralMinionsKilled,
                };
            }
        }
    }
}
```

```

        GoldEarned = p.GoldEarned,
        SummonerName = string.IsNullOrEmpty(p.RiotIdGameName) ? p.SummonerName : p.I
    };

    var player = new Player
    {
        SummonerName = string.IsNullOrEmpty(p.RiotIdGameName) ? p.SummonerName : p.I
        Champion = champion,
        Kills = p.Kills,
        Deaths = p.Deaths,
        Assists = p.Assists,
        CS = p.TotalMinionsKilled + p.NeutralMinionsKilled,
        GoldEarned = p.GoldEarned,
        Win = p.Win,
        Role = ParseRole(p.IndividualPosition)
    };

    if (p.TeamId == 100)
    {
        player.Team = team100;
        team100.Players.Add(player);
        team100.Win = p.Win;
    }
    else
    {
        player.Team = team200;
        team200.Players.Add(player);
        team200.Win = p.Win;
    }

    match.Players.Add(player);
}

match.Teams.Add(team100);
match.Teams.Add(team200);

return match;
}

private ChampionRole ParseRole(string position) => position switch
{
    "TOP" => ChampionRole.Top,
    "JUNGLE" => ChampionRole.Jungle,
    "MIDDLE" => ChampionRole.Mid,
    "BOTTOM" => ChampionRole.ADC,
    "UTILITY" => ChampionRole.Support,
    _ => ChampionRole.Unknown
};
}
}

```

9. Controllers/PostMatchController.cs

```

using Microsoft.AspNetCore.Mvc;
using PostMatchSummary.Services;

namespace PostMatchSummary.Controllers
{
    public class PostMatchController : Controller
    {
    }
}

```

```

{
    private readonly RiotService _riotService;

    public PostMatchController(RiotService riotService)
    {
        _riotService = riotService;
    }

    public async Task<IActionResult> Index(string matchId)
    {
        var match = await _riotService.GetMatchAsync(matchId ?? "EUW1_7796654199");

        if (match == null)
            return Content("Greška pri dohvaćanju matcha");

        // LINQ upiti nad našim modelom
        var topKiller = match.Players.OrderByDescending(p => p.Kills).FirstOrDefault();
        var longMatches = match.Duration > 1800;
        ViewBag.TopKiller = topKiller;
        ViewBag.IsLongMatch = longMatches;

        return View(match);
    }
}

```

10. Views/PostMatch/Index.cshtml

```

@model PostMatchSummary.Models.Match

<h2>Match: @Model.MatchId</h2>
<p>Trajanje: @(Model.Duration / 60) min | Mode: @Model.GameMode | Datum: @Model.GameCreation.To

@if (ViewBag.TopKiller != null)
{
    <p>☑ Top killer: <strong>@ViewBag.TopKiller.SummonerName</strong> (@ViewBag.TopKiller.Champ
}

@foreach (var team in Model.Teams)
{
    <h3>@team.TeamName – @(team.Win ? "✔ Pobjeda" : "✖ Poraz") | Ukupno killova: @team.TotalKill
    <table border="1" cellpadding="6">
        <thead>
            <tr>
                <th>Igrač</th>
                <th>Champion</th>
                <th>Rola</th>
                <th>K/D/A</th>
                <th>CS</th>
                <th>Gold</th>
            </tr>
        </thead>
        <tbody>
            @foreach (var p in team.Players)
            {
                <tr>
                    <td>@p.SummonerName</td>
                    <td>@p.Champion.Name</td>
                    <td>@p.Role</td>

```

```

                <td>@p.Kills/@p.Deaths/@p.Assists</td>
                <td>@p.CS</td>
                <td>@p.GoldEarned</td>
            </tr>
        }
    </tbody>
</table>
}

```

Sada tok podataka izgleda ovako:

```

Riot API (JSON)
↓
RiotResponse (deserijalizacija)
↓
MapToMatch() → Match, Team, Player, Champion (TVOJE klase!)
↓
Controller (LINQ upiti nad Match objektom)
↓
View prikazuje Match model

```

Zamijeni sve fileove, spremi APIkey i dotnet run. ☒

Champion.cs - error - na public class Champion "namespace already contains a definition for champion"

Team.cs - error - na public List<Player> "type or namespace "player" could not be found"

Match.cs -> isto kao team.cs

MatchSummary.cs isto kao ovo dvoje

RiotServices

```

var champion = new Champion
{
    Name = p.ChampionName,
    Role = ParseRole(p.IndividualPosition),
    Kills = p.Kills,
    Deaths = p.Deaths,
    Assists = p.Assists,
    CS = p.TotalMinionsKilled + p.NeutralMinionsKilled,
    GoldEarned = p.GoldEarned,
    SummonerName = string.IsNullOrEmpty(p.RiotIdGameName) ? p.SummonerName :
p.RiotIdGameName
};

```

ovo nista ne radi

var player = new player isto error

Svi errori dolaze iz istog uzroka — klase nemaju using ili su u krivom namespaceu. Evo ispravnih verzija:

Models/Champion.cs

```
namespace PostMatchSummary.Models
{
    public class Champion
    {
        public string Name { get; set; } = string.Empty;
        public ChampionRole Role { get; set; }
        public int Kills { get; set; }
        public int Deaths { get; set; }
        public int Assists { get; set; }
        public int CS { get; set; }
        public int GoldEarned { get; set; }
        public string SummonerName { get; set; } = string.Empty;
    }
}
```

Models/ChampionRole.cs

```
namespace PostMatchSummary.Models
{
    public enum ChampionRole
    {
        Top,
        Jungle,
        Mid,
        ADC,
        Support,
        Unknown
    }
}
```

Models/Player.cs

```
namespace PostMatchSummary.Models
{
    public class Player
    {
        public string SummonerName { get; set; } = string.Empty;
        public Champion Champion { get; set; } = new();
        public int Kills { get; set; }
        public int Deaths { get; set; }
        public int Assists { get; set; }
        public int CS { get; set; }
        public int GoldEarned { get; set; }
        public bool Win { get; set; }
        public ChampionRole Role { get; set; }
        public Team? Team { get; set; }
    }
}
```

Models/Team.cs

```
namespace PostMatchSummary.Models
{
    public class Team
    {
        public int TeamId { get; set; }
        public string TeamName => TeamId == 100 ? "Blue Team" : "Red Team";
        public bool Win { get; set; }
        public List<Player> Players { get; set; } = new();
        public int TotalKills => Players.Sum(p => p.Kills);
        public int TotalGold => Players.Sum(p => p.GoldEarned);
    }
}
```

Models/Match.cs

```
namespace PostMatchSummary.Models
{
    public class Match
    {
        public string MatchId { get; set; } = string.Empty;
        public DateTime GameCreation { get; set; }
        public int Duration { get; set; }
        public string GameMode { get; set; } = string.Empty;
        public string GameVersion { get; set; } = string.Empty;
        public List<Player> Players { get; set; } = new();
        public List<Team> Teams { get; set; } = new();
        public Team? WinnerTeam => Teams.FirstOrDefault(t => t.Win);
        public int TotalKills => Players.Sum(p => p.Kills);
    }
}
```

Models/MatchSummary.cs

```
namespace PostMatchSummary.Models
{
    public class MatchSummary
    {
        public string MatchId { get; set; } = string.Empty;
        public string WinnerTeamName { get; set; } = string.Empty;
        public int TotalKills { get; set; }
        public int DurationMinutes { get; set; }
        public string GameMode { get; set; } = string.Empty;
    }
}
```

Services/RiotServices.cs

```
using System.Text.Json;
using PostMatchSummary.Models;

namespace PostMatchSummary.Services
{
    public class RiotService
    {

```

```

private readonly HttpClient _httpClient;
private string apiKey = "RGAPI-TVOJ-KEY";

public RiotService(HttpClient httpClient)
{
    _httpClient = httpClient;
}

public async Task<Match?> GetMatchAsync(string matchId)
{
    using var request = new HttpRequestMessage(HttpMethod.Get,
        $"https://europe.api.riotgames.com/lol/match/v5/matches/{matchId}");

    request.Headers.Add("X-Riot-Token", apiKey);

    var response = await _httpClient.SendAsync(request);
    var json = await response.Content.ReadAsStringAsync();

    var options = new JsonSerializerOptions { PropertyNameCaseInsensitive = true };
    var riotResponse = JsonSerializer.Deserialize<RiotResponse>(json, options);

    if (riotResponse == null) return null;

    return MapToMatch(riotResponse);
}

private Match MapToMatch(RiotResponse riotResponse)
{
    var match = new Match
    {
        MatchId = riotResponse.Metadata.MatchId,
        Duration = riotResponse.Info.GameDuration,
        GameMode = riotResponse.Info.GameMode,
        GameVersion = riotResponse.Info.GameVersion,
        GameCreation = DateTimeOffset.FromUnixTimeMilliseconds(riotResponse.Info.GameCre
    };

    var team100 = new Team { TeamId = 100 };
    var team200 = new Team { TeamId = 200 };

    foreach (var p in riotResponse.Info.Participants)
    {
        var champion = new Champion
        {
            Name = p.ChampionName,
            Role = ParseRole(p.IndividualPosition),
            Kills = p.Kills,
            Deaths = p.Deaths,
            Assists = p.Assists,
            CS = p.TotalMinionsKilled + p.NeutralMinionsKilled,
            GoldEarned = p.GoldEarned,
            SummonerName = string.IsNullOrEmpty(p.RiotIdGameName) ? p.SummonerName : p.I
        };

        var player = new Player
        {
            SummonerName = string.IsNullOrEmpty(p.RiotIdGameName) ? p.SummonerName : p.I
            Champion = champion,
            Kills = p.Kills,
            Deaths = p.Deaths,
            Assists = p.Assists,
            CS = p.TotalMinionsKilled + p.NeutralMinionsKilled,

```

```

        GoldEarned = p.GoldEarned,
        Win = p.Win,
        Role = ParseRole(p.IndividualPosition)
    };

    if (p.TeamId == 100)
    {
        player.Team = team100;
        team100.Players.Add(player);
        team100.Win = p.Win;
    }
    else
    {
        player.Team = team200;
        team200.Players.Add(player);
        team200.Win = p.Win;
    }

    match.Players.Add(player);
}

match.Teams.Add(team100);
match.Teams.Add(team200);

return match;
}

private ChampionRole ParseRole(string position) => position switch
{
    "TOP" => ChampionRole.Top,
    "JUNGLE" => ChampionRole.Jungle,
    "MIDDLE" => ChampionRole.Mid,
    "BOTTOM" => ChampionRole.ADC,
    "UTILITY" => ChampionRole.Support,
    _ => ChampionRole.Unknown
};
}
}

```

Ključna stvar — svaki file mora imati točno isti namespace `PostMatchSummary.Models`. Problem je bio što `ChampionRole.cs` nije imao namespace pa je bio u globalnom scopeu i konfliktao s `Champion.cs`. `dotnet run` nakon svih promjena. ☒

Match:

Trajanje: 0 min | Mode: | Datum: 01.01.1970

Blue Team — ✖ Poraz | Ukupno killova: 0 | Ukupno golda: 0

Igrač `ChampionRolaK/D/ACSGold`

Red Team — ✖ Poraz | Ukupno killova: 0 | Ukupno golda: 0

Igrač `ChampionRolaK/D/ACSGold`

okej sada dobijem ovaj prikaz kada odem na <http://localhost:5044/PostMatch> jer je `matchid` hardcoded trenutno

Problem je što `GetMatchAsync` vraća `Match`? ali controller još uvijek poziva `GetRawJsonAsync`. Provjeri controller — mora pozivati `GetMatchAsync`, ne `GetRawJsonAsync`.

Controllers/PostMatchController.cs — provjeri da je OVAKO:

```
using Microsoft.AspNetCore.Mvc;
using PostMatchSummary.Services;

namespace PostMatchSummary.Controllers
{
    public class PostMatchController : Controller
    {
        private readonly RiotService _riotService;

        public PostMatchController(RiotService riotService)
        {
            _riotService = riotService;
        }

        public async Task<IActionResult> Index(string matchId)
        {
            var match = await _riotService.GetMatchAsync(matchId ?? "EUW1_7796654199");

            if (match == null)
                return Content("Greška pri dohvaćanju matcha");

            var topKiller = match.Players.OrderByDescending(p => p.Kills).FirstOrDefault();
            ViewBag.TopKiller = topKiller;

            return View(match);
        }
    }
}
```

Datum 01.01.1970 i prazni podaci znače da MapToMatch dobiva prazan RiotResponse — što znači da deserijalizacija faila tiho jer se poziva stara metoda koja vraća krivi tip.

Nakon promjene Ctrl+C → dotnet run → <http://localhost:5044/PostMatch> — trebao bi vidjeti prave podatke.☒

namespace PostMatchSummary.Models

```
{
    public class MatchSummary
    {
        public string MatchId {get;set;} = string.Empty;
        public string WinnerTeamName {get;set;} = string.Empty;
        public int TotalKills {get;set;}
        public int DurationMinutes {get;set;}
        public string GameMode {get;set;} = string.Empty;
    }
}
```

Answer skipped.

namespace PostMatchSummary.Models

```
{
    public class MatchSummary
    {
        public string MatchId {get;set;} = string.Empty;
        public string WinnerTeamName {get;set;} = string.Empty;
        public int TotalKills {get;set;}
        public int DurationMinutes {get;set;}
        public string GameMode {get;set;} = string.Empty;
    }
}
```

se ne koristi nigdje, idemo i to implementat

MatchSummary ima smisla koristiti kao sažetak **matcha** koji controller kreira iz Match objekta i prosljeđuje viewu kao dodatni podatak.

Controllers/PostMatchController.cs

```
using Microsoft.AspNetCore.Mvc;
using PostMatchSummary.Services;
using PostMatchSummary.Models;

namespace PostMatchSummary.Controllers
{
    public class PostMatchController : Controller
    {
        private readonly RiotService _riotService;

        public PostMatchController(RiotService riotService)
        {
            _riotService = riotService;
        }

        public async Task<IActionResult> Index(string matchId)
        {
            var match = await _riotService.GetMatchAsync(matchId ?? "EUW1_7796654199");

            if (match == null)
                return Content("Greška pri dohvaćanju matcha");

            // Kreiramo MatchSummary iz Match objekta
            var summary = new MatchSummary
            {
                MatchId = match.MatchId,
                WinnerTeamName = match.WinnerTeam?.TeamName ?? "Nepoznato",
                TotalKills = match.TotalKills,
                DurationMinutes = match.Duration / 60,
                GameMode = match.GameMode
            };

            // LINQ upiti
            var topKiller = match.Players.OrderByDescending(p => p.Kills).FirstOrDefault();
            var topCS = match.Players.OrderByDescending(p => p.CS).FirstOrDefault();
            var bestKDA = match.Players
                .Where(p => p.Deaths > 0)
                .OrderByDescending(p => (double)(p.Kills + p.Assists) / p.Deaths)
```

```

        .FirstOrDefault();

        ViewBag.Summary = summary;
        ViewBag.TopKiller = topKiller;
        ViewBag.TopCS = topCS;
        ViewBag.BestKDA = bestKDA;

        return View(match);
    }
}

```

Views/PostMatch/Index.cshtml — dodaj summary na vrh

```

@model PostMatchSummary.Models.Match

@{
    var summary = ViewBag.Summary as PostMatchSummary.Models.MatchSummary;
}

@* MATCH SUMMARY *@
<div style="background:#f0f0f0; padding:12px; margin-bottom:16px; border-radius:6px">
    <h2>Match: @summary?.MatchId</h2>
    <p>☒ Trajanje: <strong>@summary?.DurationMinutes min</strong> |
        ☒ Mode: <strong>@summary?.GameMode</strong> |
        ☒ Datum: <strong>@Model.GameCreation.ToString("dd.MM.yyyy HH:mm")</strong></p>
    <p>☒ Pobjednik: <strong>@summary?.WinnerTeamName</strong> |
        ☒ Ukupno killova: <strong>@summary?.TotalKills</strong></p>
</div>

@* MVP STATISTIKE *@
<div style="margin-bottom:16px">
    @if (ViewBag.TopKiller != null)
    {
        <p>☒ Top Killer: <strong>@ViewBag.TopKiller.SummonerName</strong> (@ViewBag.TopKiller.ChampionName)
    }
    @if (ViewBag.TopCS != null)
    {
        <p>☒ Top CS: <strong>@ViewBag.TopCS.SummonerName</strong> (@ViewBag.TopCS.ChampionName)
    }
    @if (ViewBag.BestKDA != null)
    {
        <p>★ Best KDA: <strong>@ViewBag.BestKDA.SummonerName</strong> (@ViewBag.BestKDA.ChampionName)
    }
</div>

@* TIMOVI *@
@foreach (var team in Model.Teams)
{
    <h3>@team.TeamName – @(team.Win ? "☑ Pobjeda" : "☒ Poraz") | Killovi: @team.TotalKills | Golovi: @team.TotalGoals
    <table border="1" cellpadding="6">
        <thead>
            <tr>
                <th>Igrač</th>
                <th>Champion</th>
                <th>Rola</th>
                <th>K/D/A</th>
                <th>CS</th>
                <th>Gold</th>
            </tr>
        </thead>
        <tbody>
            @foreach (var player in team.Players)
            {
                <tr>
                    <td>@player.SummonerName</td>
                    <td>@player.ChampionName</td>
                    <td>@player.Role</td>
                    <td>@player.Kills/@player.Deaths/@player.Assists</td>
                    <td>@player.CS</td>
                    <td>@player.Gold</td>
                </tr>
            }
        </tbody>
    </table>
}

```

```

        </tr>
    </thead>
    <tbody>
        @foreach (var p in team.Players)
        {
            <tr>
                <td>@p.SummonerName</td>
                <td>@p.Champion.Name</td>
                <td>@p.Role</td>
                <td>@p.Kills/@p.Deaths/@p.Assists</td>
                <td>@p.CS</td>
                <td>@p.GoldEarned</td>
            </tr>
        }
    </tbody>
</table>
}

```

Sada MatchSummary se kreira u controlleru iz Match objekta i koristi se za prikaz sažetka na vrhu stranice zajedno s MVP statistikama iz LINQ upita.☒

I jos mi dodaj, da mogu vidjeti raw json, isto kako imam prikaz na index,htmlcs da vidim lijepo uredeno, tako mi daj i uvid u raw json

Dodaj novu akciju u controller i novi view za raw JSON prikaz.

Controllers/PostMatchController.cs — dodaj RawJson akciju

```

using Microsoft.AspNetCore.Mvc;
using PostMatchSummary.Services;
using PostMatchSummary.Models;
using System.Text.Json;

namespace PostMatchSummary.Controllers
{
    public class PostMatchController : Controller
    {
        private readonly RiotService _riotService;

        public PostMatchController(RiotService riotService)
        {
            _riotService = riotService;
        }

        public async Task<IActionResult> Index(string matchId)
        {
            var match = await _riotService.GetMatchAsync(matchId ?? "EUW1_7796654199");

            if (match == null)
                return Content("Greška pri dohvaćanju matcha");

            var summary = new MatchSummary
            {
                MatchId = match.MatchId,
                WinnerTeamName = match.WinnerTeam?.TeamName ?? "Nepoznato",
                TotalKills = match.TotalKills,

```



```

        gap: 16px;
    }
    .toolbar a {
        color: #4fc3f7;
        text-decoration: none;
        padding: 6px 14px;
        border: 1px solid #4fc3f7;
        border-radius: 4px;
    }
    .toolbar a:hover { background: #4fc3f7; color: #1e1e1e; }
    .toolbar span { color: #aaa; font-size: 14px; }

    pre {
        background: #252526;
        padding: 20px;
        border-radius: 6px;
        overflow-x: auto;
        line-height: 1.6;
        font-size: 13px;
        border: 1px solid #3c3c3c;
    }

    /* Syntax highlighting */
    .json-key    { color: #9cdcfe; }
    .json-string { color: #ce9178; }
    .json-number { color: #b5cea8; }
    .json-bool   { color: #569cd6; }
    .json-null   { color: #569cd6; }
</style>
</head>
<body>
    <div class="toolbar">
        <a href="/PostMatch?matchId=@ViewBag.MatchId">← Natrag na prikaz</a>
        <span>Raw JSON za match: <strong style="color:white">@ViewBag.MatchId</strong></span>
        <button onclick="copyJson()" style="background:#4caf50;color:white;border:none;padding:6px">
    </div>

    <pre id="jsonContent">@Html.Raw(HighlightJson(Model))</pre>

    <script>
        function copyJson() {
            const text = @Html.Raw(System.Text.Json.JsonSerializer.Serialize(Model));
            navigator.clipboard.writeText(text).then(() => alert('JSON kopiran!'));
        }
    </script>
</body>
</html>

@functions {
    string HighlightJson(string json)
    {
        // Syntax highlight JSON
        var result = System.Text.RegularExpressions.Regex.Replace(
            System.Web.HttpUtility.HtmlEncode(json),
            @"\"\"(.*)\"\"\"\"\\s*:",
            "<span class='json-key'>\\\"$1\\\"</span>:"
        );
        result = System.Text.RegularExpressions.Regex.Replace(
            result,
            @":\\s*\"\"(.*)\"\"\"",
            ": <span class='json-string'>\\\"$1\\\"</span>"
        );
    }
}

```

```

        result = System.Text.RegularExpressions.Regex.Replace(
            result,
            @"":\s*(\d+\.\d*)",
            "": <span class='json-number'>$1</span>"
        );
        result = System.Text.RegularExpressions.Regex.Replace(
            result,
            @"":\s*(true|false)",
            "": <span class='json-bool'>$1</span>"
        );
        result = System.Text.RegularExpressions.Regex.Replace(
            result,
            @"":\s*(null)",
            "": <span class='json-null'>$1</span>"
        );
        return result;
    }
}

```

Dodaj link u Index.cshtml — na vrh stranice

```

<div style="margin-bottom:12px">
    <a href="/PostMatch/RawJson?matchId=@Model.MatchId"
        style="padding:6px 14px; background:#333; color:white; text-decoration:none; border-radius:
        ☒ Pogledaj Raw JSON
    </a>
</div>

```

Sada imaš:

- <http://localhost:5044/PostMatch> → lijepi tablični prikaz
- <http://localhost:5044/PostMatch/RawJson> → syntax-highlighted JSON s dark temom i gumbom za kopiranje ☒

public async Task<IActionResult> RawJson(string matchId)

```

{
    var json = await _riotService.GetRawJsonAsync(matchId ?? "EUW1_7796654199");

    // Pretty print JSON
    var parsed = JsonSerializer.Deserialize<object>(json);
    var prettyJson = JsonSerializer.Serialize(parsed, new JsonSerializerOptions
    {
        WriteIndented = true
    });

    ViewBag.MatchId = matchId ?? "EUW1_7796654199";
    return View("RawJson", prettyJson);
}

```

ovaj dio nemam u riot services

GetRawJsonAsync metoda već postoji u RiotServices.cs — provjeri je li još uvijek tamo. Trebala bi biti na dnu klase:

Services/RiotServices.cs — provjeri da imaš OBE metode:

```
using System.Text.Json;
using PostMatchSummary.Models;

namespace PostMatchSummary.Services
{
    public class RiotService
    {
        private readonly HttpClient _httpClient;
        private string apiKey = "RGAPI-TV0J-KEY";

        public RiotService(HttpClient httpClient)
        {
            _httpClient = httpClient;
        }

        public async Task<Match?> GetMatchAsync(string matchId)
        {
            using var request = new HttpRequestMessage(HttpMethod.Get,
                $"https://europe.api.riotgames.com/lol/match/v5/matches/{matchId}");

            request.Headers.Add("X-Riot-Token", apiKey);

            var response = await _httpClient.SendAsync(request);
            var json = await response.Content.ReadAsStringAsync();

            var options = new JsonSerializerOptions { PropertyNameCaseInsensitive = true };
            var riotResponse = JsonSerializer.Deserialize<RiotResponse>(json, options);

            if (riotResponse == null) return null;

            return MapToMatch(riotResponse);
        }

        // ← OVA METODA treba biti tu
        public async Task<string> GetRawJsonAsync(string matchId)
        {
            using var request = new HttpRequestMessage(HttpMethod.Get,
                $"https://europe.api.riotgames.com/lol/match/v5/matches/{matchId}");

            request.Headers.Add("X-Riot-Token", apiKey);

            var response = await _httpClient.SendAsync(request);
            return await response.Content.ReadAsStringAsync();
        }

        private Match MapToMatch(RiotResponse riotResponse)
        {
            var match = new Match
            {
                MatchId = riotResponse.Metadata.MatchId,
                Duration = riotResponse.Info.GameDuration,
                GameMode = riotResponse.Info.GameMode,
                GameVersion = riotResponse.Info.GameVersion,
                GameCreation = DateTimeOffset.FromUnixTimeMilliseconds(riotResponse.Info.GameCre
            };
        }
    }
}
```

```

var team100 = new Team { TeamId = 100 };
var team200 = new Team { TeamId = 200 };

foreach (var p in riotResponse.Info.Participants)
{
    var champion = new Champion
    {
        Name = p.ChampionName,
        Role = ParseRole(p.IndividualPosition),
        Kills = p.Kills,
        Deaths = p.Deaths,
        Assists = p.Assists,
        CS = p.TotalMinionsKilled + p.NeutralMinionsKilled,
        GoldEarned = p.GoldEarned,
        SummonerName = string.IsNullOrEmpty(p.RiotIdGameName) ? p.SummonerName : p.I
    };

    var player = new Player
    {
        SummonerName = string.IsNullOrEmpty(p.RiotIdGameName) ? p.SummonerName : p.I
        Champion = champion,
        Kills = p.Kills,
        Deaths = p.Deaths,
        Assists = p.Assists,
        CS = p.TotalMinionsKilled + p.NeutralMinionsKilled,
        GoldEarned = p.GoldEarned,
        Win = p.Win,
        Role = ParseRole(p.IndividualPosition)
    };

    if (p.TeamId == 100)
    {
        player.Team = team100;
        team100.Players.Add(player);
        team100.Win = p.Win;
    }
    else
    {
        player.Team = team200;
        team200.Players.Add(player);
        team200.Win = p.Win;
    }

    match.Players.Add(player);
}

match.Teams.Add(team100);
match.Teams.Add(team200);

return match;
}

private ChampionRole ParseRole(string position) => position switch
{
    "TOP" => ChampionRole.Top,
    "JUNGLE" => ChampionRole.Jungle,
    "MIDDLE" => ChampionRole.Mid,
    "BOTTOM" => ChampionRole.ADC,
    "UTILITY" => ChampionRole.Support,
    _ => ChampionRole.Unknown
};

```

```
}  
}
```

Zamijeni cijeli file s ovim — `GetRawJsonAsync` je na liniji 18, odmah ispod `GetMatchAsync`. `dotnet run` i trebalo bi raditi. ☒

1. <https://learn.microsoft.com/en-us/visualstudio/get-started/csharp/tutorial-aspnet-core?view=visualstudio>
2. <https://learn.microsoft.com/ka-ge/dotnet/core/tools/dotnet-run>
3. image-7.jpg
4. image-9.jpg
5. image-6.jpg
6. image-8.jpg
7. image-10.jpg
8. <https://learn.microsoft.com/en-us/dotnet/core/tools/dotnet-run>
9. https://secretgeek.net/dotnet_run
10. <https://github.com/dotnet/docs/blob/main/docs/core/tools/dotnet-run.md>
11. <https://phaisarn.com/2019/dotnet-core-command/>
12. <https://learn.microsoft.com/en-us/answers/questions/2074809/tutorial-create-an-asp-net-core-app-with-angular-i>
13. <https://support.microsoft.com/hr-hr/office/omogu%C4%87ite-gostima-da-vidite-va%C5%A1-projekt-u-project-za-web-81e1c210-74b2-4711-98ab-dea58f1b566b>
14. <https://www.youtube.com/watch?v=7b9l2J6U5lY>
15. <https://support.microsoft.com/hr-hr/office/pokretanje-prezentacije-i-pregledavanje-bilje%C5%A1ki-u-prikazu-izlaga%C4%87a-4de90e28-487e-435c-9401-eb49a3801257>
16. <https://softchris.github.io/pages/dotnet-10-commands.html>
17. <https://georgestocker.com/2017/01/30/how-do-i-debug-asp-net-core-1-1-applications-in-visual-studio-2015/>
18. https://web-arhiva.skole.hr/os-dpejacevic-na/upload/os-dpejacevic-na/images/static3/1218/attachment/KAKO_PRIKAZATI_PROJEKT.pdf
19. <https://stackoverflow.com/questions/58978232/can-we-use-dotnet-run-for-running-aspnet-core-application-without-project-file>
20. <https://learn.microsoft.com/en-us/visualstudio/debugger/how-to-enable-debugging-for-aspnet-applications?view=visualstudio>
21. image.jpg
22. image-2.jpg
23. image-3.jpg
24. image-4.jpg
25. image-4.jpg
26. <https://learn.microsoft.com/ka-ge/dotnet/core/tools/dotnet-run>
27. <https://learn.microsoft.com/ka-ge/dotnet/core/tools/dotnet-run>
28. image.jpg
29. RiotServices-11.cs
30. RiotResponse-9.cs
31. Team-10.cs

32. ViewImports-12.cshtml
33. ViewStart-13.cshtml
34. Privacy-15.cshtml
35. Index-15.cshtml
36. Layout.cshtml-17.css
37. Error-19.cshtml
38. Layout-16.cshtml
39. PostMatchController-2.cs
40. HomeController.cs
41. Champion-3.cs
42. ChampionRole-4.cs
43. Match-6.cs
44. ErrorViewModel-5.cs
45. MatchSummary-7.cs
46. Player-8.cs
47. RiotServices-11.cs
48. RiotResponse-9.cs
49. ErrorViewModel-5.cs
50. ChampionRole-4.cs
51. Match-6.cs
52. MatchSummary-7.cs
53. Team-10.cs
54. Player-8.cs
55. Champion-3.cs
56. Index-15.cshtml
57. PostMatchController-2.cs
58. Layout-16.cshtml
59. image.jpg
60. image.jpg
61. image.jpg
62. <https://darkintaqt.com/blog/spectator-v4>
63. <https://gist.github.com/RiotTuxedo/758ee4d88693b768a880ece93cd78663>
64. <https://github.com/RiotGames/developer-relations/issues/754>
65. <https://www.itero.gg/articles/riot-api>
66. <https://developer.riotgames.com>
67. https://romg2.github.io/opgg/01_Day01_02_GET_match_stats/
68. https://www.reddit.com/r/leagueoflegends/comments/1gui15s/riot_api_question/
69. https://www.reddit.com/r/leagueoflegends/comments/lynats/i_played_over_2000_aram_games_and_used_riots_api/
70. <https://developer.riotgames.com/apis>
71. <https://github.com/RiotGames/developer-relations/issues/801>
72. <https://mintlify.com/mangeloni30/agent-lol/api/matches>

73. <https://riot-watcher.readthedocs.io/en/latest/riotwatcher/LeagueOfLegends/MatchApiV5.html>

74. <https://pkg.go.dev/github.com/yigithanbalci/golio/riot/lol>

75. <https://stackoverflow.com/questions/75828268/so-i-have-a-huge-json-file-and-i-want-to-create-a-smaller-version-of-it-where-on>

76. <https://stackoverflow.com/questions/71274769/keyerror-with-riot-api-matchv5-when-trying-to-pull-data>

77. [Lab-1-2.md](#)